Knowledge-Based Systems 323 (2025) 113720

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

# DFGCN: Decoupled dual-flow dynamic graph convolutional network for multivariate time series forecasting

Junjie Ye [a], Jinhong Li [a], Rui Su [b], Sen Yang [c], Yaqun Huang [a], Chunna Zhao [a,*]

[a] School of Information Science & Engineering, Yunnan University, Kunming, Yunnan, 650504, China
[b] Faculty of Information Engineering and Automation, Kunming University of Science and Technology, Kunming, Yunnan, 650500, China
[c] Department of Computer Science and Technology, West Yunnan University of Applied Sciences, Dali, Yunnan, 671000, China

ARTICLE INFO

*Keywords:*
Multivariate time series forecasting
Dynamic sparse graph neural network
Dual-flow decoupling
Intra- and inter-series dependencies
Heterogeneity

ABSTRACT

Multivariate time series forecasting (MTSF) is of critical importance in various domains, including climate prediction, financial analysis, and energy management. Multivariate time series data typically exhibit both intra-series and inter-series dependencies, leading to intricate and intertwined relationships. Recently, numerous studies have focused on capturing both types of correlations. Most existing methods adopt a single-flow structure, attempting to capture features from both the time and variable dimensions simultaneously. However, due to the heterogeneous dependencies between the time dimension and variable dimension, single-flow structures are prone to interference in feature learning, making it difficult to effectively model complex dependencies. To address this issue, we propose a decoupled **D**ual-**F**low dynamic **G**raph **C**onvolutional **N**etwork (DFGCN) for MTSF, which separately models the intra-series and inter-series dependencies through decoupled feature learning. First, a dual-flow embedding layer is employed to separately represent the features of the time and variable dimensions. Next, we propose a dual-flow dynamic sparse GCN incorporating Pearson correlation, which robustly learns both intra-series and inter-series dynamic dependencies, while effectively reducing the computational cost of the GCN. Then, we introduce a dual-flow global attention layer, which separately learns the global information of inter-series and intra-series dependencies. By combining dynamic sparse GCN and the global attention mechanism, DFGCN is able to effectively model both local and global features in multivariate time series data. Experimental results on multiple real-world datasets show that DFGCN outperforms strong baseline models, with improvements ranging from 2.40% to 19.84%. Ablation experiments further validate the model's robustness and the effectiveness of its components. Code is available at this repository: https://github.com/junjieyePhD/DFGCN.

## 1. Introduction

Time series forecasting [1–3] is widely applied in various fields, including traffic flow prediction [4], climate change analysis [5], disease spread monitoring [6], and financial market forecasting [7]. Multivariate time series forecasting (MTSF) [8,9] is an important research direction in deep learning. It aims to predict future trends by analyzing the potential patterns in multiple historical variables, and has become a focal point for many researchers. Multivariate time series data contains multiple variables [10,11], which may exhibit complex interdependencies. Therefore, in MTSF tasks, it is essential not only to focus on the temporal trend of individual variables, but also to consider the interactions and influences between variables [12], in order to more accurately capture the spatiotemporal dependencies and latent patterns within the data. This makes MTSF an important and challenging direction in the field of machine learning.

In MTSF tasks, traditional statistical methods such as autoregressive models [13] and moving average models [14] have gradually become inadequate for meeting the modeling demands of large-scale data and nonlinear relationships. In recent years, deep learning methods have made significant progress in time series forecasting tasks, especially models based on Graph Neural Networks (GNN) [15,16], Convolutional Neural Networks (CNN) [17,18], Multi-Layer Perceptrons (MLP) [19,20], and Transformer architectures with self-attention mechanisms [21,22]. Due to their powerful modeling capabilities and high flexibility, these methods are widely used in MTSF. In summary, existing approaches for MTSF can be broadly categorized into

* Corresponding author.
*E-mail addresses:* yejunjie@stu.ynu.edu.cn (J. Ye), jhli@mail.ynu.edu.cn (J. Li), sr@stu.kust.edu.cn (R. Su), 1240040@wyuas.edu.cn (S. Yang), huangyq@ynu.edu.cn (Y. Huang), zhaochunna@ynu.edu.cn (C. Zhao).

https://doi.org/10.1016/j.knosys.2025.113720
Received 16 January 2025; Received in revised form 23 April 2025; Accepted 6 May 2025
Available online 22 May 2025
0950-7051/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

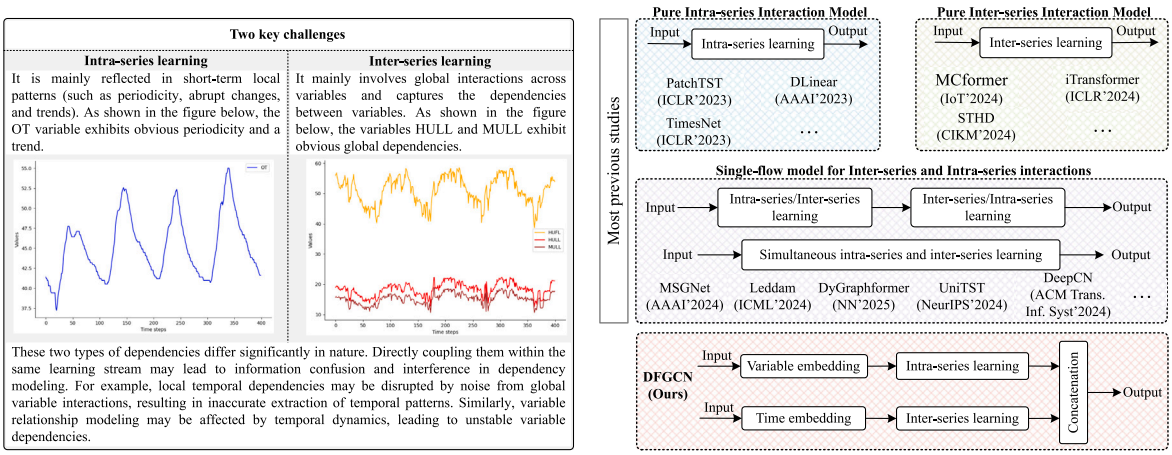


**Fig. 1.** Motivation for decoupling in dual-flow network. Most existing studies adopt a single-flow learning strategy, coupling inter- and intra-series dependencies within the same learning process. However, this approach often leads to information confusion and hinders effective dependency modeling. OT, HULL, MULL, and HUFL denote variables from the ETTh1 dataset.

two aspects: inter-series modeling and intra-series modeling, as illustrated in Fig. 1. Inter-series modeling aims to learn the interdependencies among different variables and capture cross-series interaction patterns, such as iTransformer [23], STHD [24], and MCformer [25]. Intra-series modeling focuses on the temporal dependencies within a single time series to extract local or global temporal dynamics, as exemplified by PatchTST [26], DLinear [27], and TimesNet [18]. Recent studies demonstrate that jointly considering both inter-series and intra-series dependencies is key to fully exploiting temporal information and enhancing forecasting performance, as shown in models like MSGNet [12], Leddam [28], DyGraphformer [29], UniTST [30], and DeepCN [31]. However, most of these methods are based on a single-flow structure, which faces a significant challenge: the dependencies along the temporal and variable dimensions are inherently heterogeneous. As illustrated on the left of Fig. 1, temporal dependencies typically appear as local dynamic variations (such as seasonality, trends, and abrupt changes), while variable dependencies often involve global cross-series interactions (including HULL and MULL global dependencies). Due to the inherently different nature of these two types of dependencies, single-flow structures struggle to model them effectively at the same time, resulting in information entanglement and limited expressiveness in dependency representation during feature learning. To address this issue, we propose a dual-flow architecture that aims to decouple intra-series and inter-series dependencies by separately modeling features along the temporal and variable dimensions, thereby avoiding information confusion and interference.

The main existing approaches for modeling inter-series dependencies include Transformer-based models, CNN-based models, and GNN-based models. Among them, the GNN-based model demonstrates great potential in handling structured data, especially in capturing dependencies within time series data. In recent years, GNN-based methods [32] for time series forecasting mainly include the Graph Convolutional Network (GCN), the graph attention network, and the spatio-temporal graph network. The graph attention network models [33] dependencies by adaptively assigning weights to neighboring nodes. However, it may suffer from high computational costs and limited capability in capturing long-range dependencies in time series modeling. The spatio-temporal graph network [34] is mainly applied to spatio-temporal forecasting tasks and typically relies on data with an explicitly defined graph structure. However, in MTSF tasks, the relationships among variables are often implicit and dynamic [35], without a fixed graph structure. In contrast, the GCN, as a representative type of GNN, aggregates information from neighboring nodes through convolution operations, thus enabling more effective modeling of inter-node dependencies. Therefore, using the GCN for dependency modeling is a reasonable choice. However, most existing GNN-based methods [36] for MTSF adopt a fully connected graph, which assumes that all variables are directly related to each other. This approach faces two main challenges: (1) high computational complexity as the number of variables increases, significantly raising the computational cost of a fully connected graph, and (2) information redundancy and noise interference, as in real-world scenarios, not all time series exhibit explicit correlations, and constructing a fully connected graph may introduce noise from irrelevant variables, degrading model performance. Therefore, it is crucial to design a dynamic sparse graph structure based on inter-series correlations to address this issue.

In summary, to address the issues mentioned above, we propose a novel model, the decoupled **D**ual-**F**low dynamic sparse **G**raph **C**onvolutional **N**etwork (DFGCN), aimed at separately modeling the dependencies between the time and variable dimensions. Specifically, we first adopt a dual-flow embedding layer to represent features along the time and variable dimensions, respectively. Next, we construct dynamic sparse graphs based on the Pearson correlation between series and employ a dual-flow graph convolutional network to learn the local dependencies both between sequences and within sequences. Then, we introduce a dual-flow global attention module to capture the global information of inter-series and intra-series dependencies, respectively. In summary, by decoupling feature learning, the DFGCN model can independently capture intra-series (time dimension) and inter-series (variable dimension) correlations, thereby enhancing prediction accuracy. Specifically, our contributions are as follows:

- We propose the DFGCN, which tackles the heterogeneity issue between the time and variable dimensions by independently learning their dependencies. This approach facilitates more effective modeling of the complex dependencies within multivariate time series data.
- We introduce a dynamic sparse graph convolutional network (DSGCN) grounded in Pearson correlation coefficients, which significantly reduces computational overhead by dynamically constructing a sparse graph based on the correlations among time series, in stark contrast to conventional fully connected graph structures. Additionally, the proposed DSGCN effectively mitigates the noise introduced by irrelevant variables, thereby enhancing both the overall performance and robustness of the model.
- Extensive evaluations on 11 real-world MTSF datasets validate the effectiveness of DFGCN. Specifically, compared with 10 state-of-the-art models across varying prediction lengths, DFGCN achieves top-1 performance in 54 experimental settings. Furthermore, additional ablation experiments further demonstrate the robustness of the DFGCN model and the effectiveness of its individual components.

## 2. Related works

### 2.1. Multivariate time series forecasting

MTSF is widely applied in fields such as finance, climate, and traffic [4,5,7]. As multivariate time series data typically involves multiple correlated variables, the inherent dependencies of multivariate time series data include intra-series dependencies and inter-series dependencies. Therefore, effectively modeling these dependencies is a core challenge in MTSF. In recent years, deep learning-based models such as Transformer [37] and GCN [38] have made significant progress in MTSF, which primarily address the challenges from three perspectives: intra-series methods, inter-series methods, and integration of inter-series and intra-series methods.

**Intra-series methods:**

Intra-series modeling focuses on capturing dependencies along the temporal dimension, such as periodicity and trends within univariate sequences. Transformer models have recently demonstrated remarkable advantages in modeling long-range dependencies, spurring numerous approaches that significantly improve long-sequence forecasting. Some studies enhance temporal modeling from a frequency-domain perspective. For instance, FEDformer [39] performs seasonal-trend decomposition and incorporates Fourier transform to enrich frequency representations, improving both global and local structure modeling. Similarly, Quatformer [40] represents complex periodic patterns via quaternion rotation attention and trend normalization, leveraging a globally decoupled memory design to better model non-stationary sequences. Other works focus on structural improvements to Transformer architectures. GCformer [41] introduces structured global convolution kernels to address self-attention's limitations in capturing local dependencies, enabling joint modeling of short- and long-term patterns. Triformer [42] incorporates a triangular attention framework and variable-specific mechanisms to learn multi-scale periodic features, enhancing adaptability to heterogeneous time series. DifFormer [43] adopts a multi-resolution perspective through differencing and lag-aware dynamic range operations, effectively capturing periodic and cyclic changes. Additionally, PatchTST [26] segments sequences into independent patches and employs channel-independent attention to reduce redundant interference and better capture local patterns. In summary, these methods explore intra-series modeling from diverse perspectives.

**Inter-series methods:** Inter-series methods primarily focus on modeling the correlations among different variables, aiming to fully exploit the latent interactions between them. To capture complex multivariate dependencies, recent studies employ strategies such as attention mechanisms, time-frequency fusion, and architectural innovations, resulting in a variety of effective sequence modeling approaches. For instance, JTFT [22] fuses time-domain and frequency-domain features by utilizing learnable frequency representations to model multi-scale dependencies, while adopting a low-rank attention mechanism to enhance the efficiency of cross-variable modeling and the expressiveness of local correlations. Similarly, Client [44] emphasizes the modeling of trend features across variables by employing a linear module to extract long-term trends and integrating attention mechanisms to capture cross-channel collaborative variations. From a structural perspective, iTransformer [23] adopts a minimalist approach by treating each variable directly as a token, thereby focusing on capturing inter-variable dependencies through attention mechanisms. STHD [24], designed for high-dimensional multivariate time series modeling, proposes a scalable Transformer framework. By introducing a sparse relation matrix and a ReIndex training strategy, the model effectively captures high-dimensional channel dependencies while maintaining computational efficiency and generalization capability during training.

**Integration of inter-series and intra-series methods:** Temporal patterns and variable dependencies are both critical for enhancing forecasting performance. Recently, numerous studies have attempted to integrate intra-series and inter-series modeling mechanisms to achieve more comprehensive information capture and representation. Among them, Crossformer [35] stands out as a representative fusion modeling approach. It employs dimension-segment embeddings and a two-stage attention mechanism to jointly model dependencies across the temporal and variable dimensions from a multi-scale perspective. CARD [45] combines channel-aligned attention, a token mixing module, and a robust loss function to simultaneously capture intra-series temporal patterns and strengthen the modeling of inter-variable correlations. In terms of structural representation, SageFormer [46] introduces a graph structure to model dependencies among variables, thereby enhancing the Transformer's capability to represent temporal features and reducing information redundancy and interference. CrossGNN [47] leverages a cross-scale graph neural network architecture along with adaptive multi-scale identifiers to improve the model's robustness against noise and to simultaneously capture both short- and long-term dependencies in spatiotemporal structures. In addition, some methods attempt to integrate frequency-domain modeling with graph structure learning to achieve comprehensive information fusion. For example, MS-GNet [12] introduces frequency-domain analysis combined with an adaptive graph convolution mechanism to effectively capture cross-variable dependencies across multiple time scales, while also enhancing intra-series modeling performance through attention mechanisms. Moreover, FourierGNN [36] approaches the problem from the perspective of graph modeling by representing sliding windows as fully connected spatiotemporal graphs and constructing a hyper-variable graph, enabling unified modeling of both temporal and variable dimensions within a single graph structure.

Although the aforementioned methods have achieved significant progress in modeling both intra-series and inter-series dependencies, the dependencies along the temporal and variable dimensions exhibit inherent heterogeneity. Most existing approaches adopt a single-stream architecture, which struggles to effectively capture both types of dependencies simultaneously, leading to interference and information loss during feature learning. Therefore, we employ a dual-flow architecture to decouple inter-series and intra-series dependencies.

### 2.2. GNN-based time series forecasting method

The GNN model is first proposed in [48] and quickly becomes a powerful tool for processing graph-structured data. GNN aims to aggregate information from neighboring nodes based on the adjacency structure, thereby capturing complex dependencies among nodes. In recent years, GNN-based models have achieved remarkable progress in spatiotemporal forecasting, benefiting from the inherent graph structure in spatiotemporal data. By leveraging the graph topology, GNN effectively models spatial dependencies within time series data and is widely adopted in traffic flow forecasting tasks.

Traffic flow forecasting serves as a key task in spatiotemporal prediction, where numerous GNN-based models have been proposed. For instance, SAGDFN [49] starts from large-scale multivariate data and introduces a scalable adaptive graph diffusion mechanism to model complex spatiotemporal dynamics. HimNet [34] focuses on the inherent spatiotemporal heterogeneity in urban traffic and proposes a modeling framework that combines spatial embeddings with temporal meta-learning. HD-TTS [50] alleviates the impact of missing data through a hierarchical spatiotemporal down-sampling mechanism while enhancing the representation of dynamic spatiotemporal variations. RGDAN [51] introduces a joint graph diffusion attention and temporal attention mechanism, enabling dynamically adaptive graph structure weights. TGCRN [52] proposes a time-aware graph structure learning approach and integrates it with a graph convolutional gated recurrent network, effectively bridging spatial topologies with temporal sequence evolution.

Although multivariate time series data do not possess a fixed graph structure, the expanding application of GNN in spatiotemporal forecasting has led to the development of several GNN-based models for

MTSF, such as FourierGNN [36], MSGNet [12], and StemGNN [15]. However, existing GNN-based approaches do not effectively address the high computational complexity and information redundancy caused by fully connected graphs, nor do they fully resolve the challenge of jointly modeling inter-series and intra-series dependencies. To tackle these issues, we propose a sparsely connected graph structure dynamically constructed based on Pearson correlation coefficients.

### 2.3. Dual-flow neural networks

The dual-flow neural network is a type of architecture that enhances overall representational capacity by modeling two distinct but complementary data streams in parallel. It has been widely applied in tasks such as image recognition, video analysis, and multimodal learning. The core idea lies in explicitly decoupling heterogeneous dependencies within complex data into two semantically distinct subspaces and modeling them separately, thereby mitigating feature interference and improving learning efficiency.

In visual recognition tasks [53,54], dual-flow architectures are used to distinguish between the temporal dynamics of actions and spatial textures. For example, DSDMT [54] uses a short-span stream and a long-span stream to capture short-term local changes and long-term global dynamics, effectively mitigating the modeling challenges caused by spatiotemporal feature coupling. In image classification tasks [55, 56], dual-flow networks improve classification accuracy by separately processing low-level features and high-level semantic features. For instance, DELTA [55] proposes a dual-branch structure that combines global priors with local multi-instance features, enabling the network to concurrently focus on the macro semantics and fine details of an image, thereby enhancing generalization capability. In multimodal learning tasks [57,58], dual-flow architectures have become one of the dominant frameworks. For example, DSPT [58] addresses the image-text mismatch problem by modeling text and images separately before fusion, effectively alleviating semantic bias and representation confusion across modalities.

Inspired by the above insights, we introduce the dual-flow concept into MTSF. Multivariate time series data inherently exhibit sequential dynamics along the temporal dimension (e.g., periodicity and trends) and interaction structures along the variable dimension (e.g., co-movements and cross-dimensional dependencies). These two types of dependencies are heterogeneous in both their representation forms and modeling objectives. Jointly modeling them often leads to information interference and reduced learning efficiency. Therefore, we propose a dual-stream decoupled dynamic modeling architecture that separately captures the two core dependencies of multivariate time series from the perspectives of a temporal stream and a variable stream, thereby avoiding the entanglement between temporal information and variable interactions.

## 3. Methodology

Without loss of generality, in MTSF tasks, the aim is to predict the future values $Y = \{x_{L+1}, x_{L+2}..., x_{L+\eta}\} \in \mathbb{R}^{N\times\eta}$ of a multivariate time series using historical multivariate time series data $X = \{x_1, x_2..., x_L\} \in \mathbb{R}^{N\times L}$, where $L$ denotes the look-back window size, $\eta$ represents the prediction horizon, and $N$ denotes the number of variables (e.g., in weather forecasting: humidity, temperature, and wind speed).

In this section, we provide a detailed description of the design framework of the proposed DFGCN model. The detailed structure of DFGCN is illustrated in Fig. 2. First, to effectively distinguish the feature diversity across different dimensions, in Section 3.1, we introduce a dual-flow node embedding layer, which includes variable node embedding and time node embedding, laying the foundation for the subsequent dual-flow modeling. Then, in Section 3.2, based on the Pearson correlations between inter-series and intra-series (inter-patches) dependencies, we construct dual-flow dynamic sparse graphs, including inter-series and intra-series dynamic sparse graphs. These graphs not only explicitly capture sequence dependencies but also significantly reduce the computational cost of graph convolution. In Section 3.3, we dynamically learn the local strong dependencies inter-series and intra-series through a dual-flow multilayer GCN, enhancing the model's ability to capture complex local relationships. In Section 3.4, we further introduce a dual-flow global attention layer to capture the global implicit dependencies inter-series and intra-series, thereby strengthening the model's expressive power. Finally, in Section 3.5, we fuse the information from the time flow and variable flow through concatenation and a projection layer, achieving a comprehensive integration of the two feature streams to improve prediction accuracy and robustness. The entire framework effectively decouples the temporal and variable dependencies in MTSF, overcoming the heterogeneity of dependencies between the time and variable dimensions, and providing an innovative solution for modeling complex time series data.

### 3.1. Multivariate time series embedding

In MTSF, commonly used embeddings include time-point embedding [59], patch embedding [26], and variable embedding [23]. Time-point embedding and patch embedding primarily represent features along the time dimension but fail to capture the differences between variables. Variable embedding focuses on features along the variable dimension and reflects the relationships between variables, but it lacks the ability to capture time-dependent features such as periodicity and long-term dependencies. To address these issues, this section proposes an embedding method based on a dual-flow strategy, utilizing both variable node embedding and time node embedding to represent the diverse features of the time and variable dimensions, thereby reducing the mutual interference between time and variable dependencies. First, similar to the PatchTST model [26], we apply the RevIN [60] normalization technique to process the data and address the issue of data shift. Then, we perform embedding processing on the multivariate time series data, as described below.

#### 3.1.1. Variable node embedding

The variable node embedding aims to represent the differences between different variables in multivariate time series data. Specifically, inspired by iTransformer, each variable is treated as a node in the graph, and each variable node is mapped to a high-dimensional vector space through a linear transformation. The process is formalized as follows:

$$E^v = Embedding(X), \tag{1}$$

where $E^v = \{E_1^v, E_2^v, \dots, E_N^v\} \in \mathbb{R}^{N\times d}$ denotes the variable flow embedding, which contains $N$ embedded variates of dimension $d$. The embedding function $Embedding : \mathbb{R}^{N\times L} \to \mathbb{R}^{N\times d}$ is implemented as a linear mapping layer. In $E^v$, the superscript $v$ specifies that the embedding is associated with the variable flow. The variable node embedding uses an independent encoding approach to avoid the interference of time dimension features on the features between variables, thereby more accurately representing the diversity of multivariate features. This method helps better capture the independent relationships between variables, improving the model's expressive power for multivariate time series data.

#### 3.1.2. Time node embedding

The goal of time node embedding is to represent the characteristics along the time dimension of multivariate time series data, such as trend, seasonality, and short-term fluctuations. However, treating each individual time point as a node in the graph may introduce two potential issues: (1) The semantic information contained in a single time point is limited, and when the model focuses on pairwise interactions, it may lead to a bottleneck in information utilization;

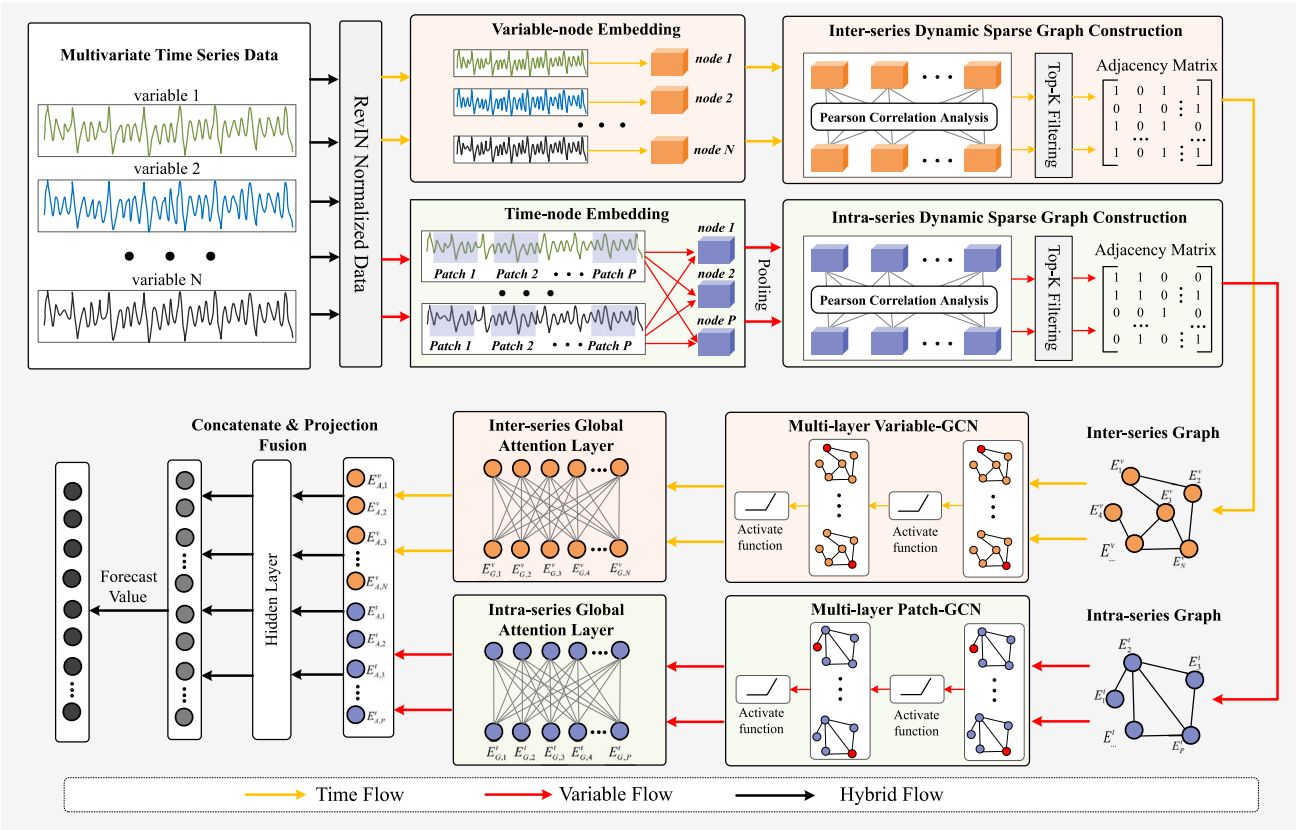


**Fig. 2.** The framework of DFGCN. The DFGCN architecture employs a dual-flow structure to decouple the time dependencies and variable dependencies in MTSF. Specifically, (1) Variable and time node embeddings are used to separately represent the features of the variable dimension and the time dimension. (2) Dynamic sparse graphs for inter-series and intra-series dependencies are constructed based on Pearson correlations between nodes. (3) Multilayer variable GCN and time GCN are used to dynamically learn the inter-variable and inter-patch dependencies. (4) A global attention layer is introduced to capture the global dependencies between variables and patches. (5) The dual-flow fusion module is proposed to integrate the information from the time flow and variable flow, including concatenation and projection operations.

(2) If each time point is treated as a node, the size of the graph will increase dramatically, significantly raising the computational cost. To address these issues, inspired by the PatchTST method, we adopt a Patch strategy to divide each sequence into $P$ subsequences, where $P = \left\lfloor \frac{L}{S} \right\rfloor + 1$, with $L$ representing the total length of the sequence and $S$ representing the length of each subsequence. In this way, we treat each subsequence as a node for time node embedding, which effectively reduces computational complexity and enhances the expressive power of the time-related features. The process is formalized as follows,

$$\{S_1, S_2, \dots, S_P\} = Patchify(X), \tag{2}$$

$$\begin{aligned} E^t &= \{E_1^t, E_2^t, \dots, E_P^t\} \\ &= PatchEmbed(S_1, S_2, \dots, S_P), \end{aligned} \tag{3}$$

where $E^t \in \mathbb{R}^{N \times P \times d}$ denotes the time flow embedding, contains $P$ embedded time segments of dimension $d$, and denotes the $i$th patch in the patch sequence. The $PatchEmbed()$ function projects each patch of length $S$ into a $d$-dimensional space with a learnable linear transformation and position embedding. In $E^t$, the superscript $t$ specifies that the embedding is associated with the time flow. The $Patchify()$ denotes the operation that segments the input sequence into subseries-level patches.

In summary, under the dual-flow architecture, variable node embedding and time node embedding separately represent the diverse features of multivariate time series data from the variable dimension and the time dimension, respectively. This separated embedding approach significantly enhances the expressive power of the data, facilitating subsequent dynamic sparse graph construction and feature learning.

### 3.2. Dual-flow dynamic sparse graph construction

The core idea of GNN is to aggregate information through the adjacency relationships in the graph structure, enabling each node to effectively utilize the features of its neighboring nodes, thereby modeling the interdependencies between nodes more efficiently. Therefore, the key to GNN modeling lies in how to construct the graph's connectivity (i.e., the adjacency matrix). In recent years, some MTSF methods based on GNN have adopted fully connected graph structures (e.g., FourierGNN [36] and FC-STGNN [11]), but this structure can lead to a significant increase in computational overhead and may treat irrelevant variables as noise, thereby impacting model performance. To address this issue, this section aims to construct intra-series dynamic sparse graphs in the time flow and inter-series dynamic sparse graphs in the variable flow, enhancing both the efficiency and accuracy of the model.

#### 3.2.1. Inter-series dynamic sparse graph

In MTSF tasks, the relationships between variables are typically data-driven, rather than relying on prior assumptions or fundamental knowledge. Therefore, to capture the interdependencies between variables, we automatically discover these relationships from the data. Specifically, as shown in Fig. 2, to effectively quantify the correlation between time series, we use Pearson correlation analysis to evaluate the similarity between variable nodes. The formula is as follows:

$$\begin{bmatrix} \rho_{1,1}, \rho_{1,2}, \dots, \rho_{1,N} \\ \rho_{2,1}, \rho_{2,2}, \dots, \rho_{2,N} \\ \vdots\ \vdots \\ \rho_{N,1}, \rho_{N,2}, \dots, \rho_{N,N} \end{bmatrix} = PCA(E^v, E^v), \tag{4}$$

$$\rho_{i,j} = \frac{\sum_{t=1}^{d} (E_i^v(t) - \overline{E_i^v})(E_j^v(t) - \overline{E_j^v})}{\sqrt{\sum_{t=1}^{d} (E_i^v(t) - \overline{E_i^v})^2}\sqrt{\sum_{t=1}^{d} (E_j^v(t) - \overline{E_j^v})^2}}, \tag{5}$$

where $PCA()$ represents the Pearson correlation analysis function. $\rho_{i,j}$ denotes the Pearson correlation coefficient between variables $i$ and $j$, and $E_i^v(t)$ and $E_j^v(t)$ denote the values of the $i$th and $j$th variables at time $t$, respectively.

Then, based on the calculated similarity weights, the top-K most relevant variable nodes are selected as adjacent nodes, thereby constructing the inter-series adjacency matrix $\mathrm{A}^v$ (inter-series sparse graph). The formula is as follows:

$$\begin{aligned} \mathrm{A}^v &\Leftarrow \begin{bmatrix} 1, 0, 1, \dots, 1 \\ 0, 1, 0, \dots, 1 \\ \vdots\vdots \\ 1, 0, 1, \dots, 1 \end{bmatrix} \\ &= Filtering(\begin{bmatrix} \rho_{1,1}, \rho_{1,2}, \dots, \rho_{1,N} \\ \rho_{2,1}, \rho_{2,2}, \dots, \rho_{2,N} \\ \vdots\vdots \\ \rho_{N,1}, \rho_{N,2}, \dots, \rho_{N,N} \end{bmatrix}), \end{aligned} \tag{6}$$

where, $Filtering()$ refers to selecting the top-K variable nodes with the highest similarity as adjacent nodes. In $\mathrm{A}^v$, the superscript $v$ indicates the inter-series sparse graph in the variable flow. In this way, the relationships between variables are discovered from the data, rather than being provided as priori knowledge.

In summary, the proposed inter-series sparse graph module can dynamically capture the time-varying dependencies between variables and construct a sparse graph structure based on their correlations. This data-driven dynamic sparse graph not only effectively reduces computational overhead but also avoids unnecessary noise interference, thereby improving the model's performance in MTSF tasks.

#### 3.2.2. Intra-series dynamic sparse graph

In MTSF tasks, modeling the time dimension is equally crucial. Just as dependencies between variables must be captured, the dependencies along the time dimension must also be dynamically learned to better reflect features such as periodicity and short-term fluctuations in the time series data. Therefore, we propose a method for constructing an intra-series dynamic sparse graph to model the dependencies between different time steps along the time dimension.

Specifically, the graph structure is determined by examining the correlations between patches, similar to the approach used in constructing the inter-series dynamic sparse graph. To effectively capture the similarity between patches, Pearson correlation analysis is first used to quantify the correlation between patches, as shown in the following formula:

$$\begin{bmatrix} \rho_{1,1}, \rho_{1,2}, \dots, \rho_{1,P} \\ \rho_{2,1}, \rho_{2,2}, \dots, \rho_{2,P} \\ \vdots\vdots \\ \rho_{P,1}, \rho_{P,2}, \dots, \rho_{P,P} \end{bmatrix} = PCA(E^t, E^t), \tag{7}$$

where $\rho_{i,j}$ represents the Pearson correlation coefficient between the $i$th patch and the $j$th patch at the time step, and $PCA()$ represents Pearson correlation analysis (Eq. (5)).

Then, based on the Patch correlation matrix calculated above, the top-K most correlated patches with each patch are selected as adjacent nodes to construct the intra-series adjacency matrix $\mathrm{A}^t$ (intra-series dynamic sparse graph). The formula is expressed as follows:

$$\begin{aligned} \mathrm{A}^t &\Leftarrow \begin{bmatrix} 1, 1, 0, \dots, 0 \\ 1, 1, 0, \dots, 1 \\ \vdots\vdots \\ 0, 0, 1, \dots, 1 \end{bmatrix} \\ &= Filtering(\begin{bmatrix} \rho_{1,1}, \rho_{1,2}, \dots, \rho_{1,P} \\ \rho_{2,1}, \rho_{2,2}, \dots, \rho_{2,P} \\ \vdots\ \vdots \\ \rho_{P,1}, \rho_{P,2}, \dots, \rho_{P,P} \end{bmatrix}), \end{aligned} \tag{8}$$

where, $Filtering()$ refers to selecting the Top-K most correlated patches with each patch from the Patch correlation matrix as adjacent nodes. In $\mathrm{A}^t$, the superscript $t$ indicates the inter-series sparse graph in the variable flow. In this way, the intra-series dynamic sparse graph not only effectively captures the dependencies between time steps, but also reduces the interference from irrelevant time steps, thereby improving the efficiency and accuracy of time dimension modeling. This method avoids the computational overhead associated with fully connected graph structures, while ensuring that the temporal and periodic features of time series data are dynamically and accurately captured.

### 3.3. Dual-flow multilayer GCN

The core idea of GCN is to aggregate the feature information of each node with its neighboring nodes' features through an adjacency matrix, thereby extracting richer node dependencies layer by layer. In this section, we utilize the inter-series and intra-series sparse graphs (i.e., adjacency matrices) constructed in Section 3.2, and employ multilayer GCN to learn the dependencies between variables and patches. Specifically, we use multilayer variable graph convolutional network (Variable-GCN) and multilayer patch graph convolutional network (Patch-GCN) to capture the spatial dependencies between variables and the dynamic dependencies along the time dimension. In this way, the model can fully learn the spatiotemporal characteristics of the data and improve the performance of MTSF.

#### 3.3.1. Multilayer variable-GCN

In the variable flow section, we employ multilayer Variable-GCN to learn the inter-series dependencies. Specifically, we construct the adjacency matrix for each layer of GCN based on the inter-series dynamic sparse graph from Section 3.2. In this section, we use variable node representations $E^v \in \mathbb{R}^{N\times d}$, where $N$ represents the number of variables, and $d$ denotes the dimensional features of the variable nodes. Then, through graph convolution operations, the node features are weighted and averaged with those of their neighboring nodes, and updated to new feature representations. The formula is expressed as follows:

$$E_G^v = \sigma(\hat{\mathrm{A}}^v E^v W^v), \tag{9}$$

$$\hat{\mathrm{A}}^v = D^{(-\frac{1}{2})}(\mathrm{A}^v + I)D^{(\frac{1}{2})}, \tag{10}$$

where $E_G^v \in \mathbb{R}^{N\times d}$ denotes the multivariate interaction features learned through GCN, and $\hat{\mathrm{A}}^v \in \mathbb{R}^{N\times N}$ denotes the normalized adjacency matrix with self-loops. The superscript $v$ denotes the variable flow. The subscript $G$ denotes the result of the GCN operation. Here, $I$ denotes the identity matrix, $D$ denotes the degree matrix, $\sigma$ is the activation function (e.g., ReLU), and $W^v$ is the learnable parameter matrix. Through the multilayer variable-GCN, the model can progressively aggregate and learn higher-order dependencies between nodes, thereby effectively modeling the spatiotemporal dependencies between variables.

#### 3.3.2. Multilayer patch-GCN

In the time flow section, we use multi-layer Patch-GCN to learn the dependencies between patches. Similar to the variable flow learning, the time flow leverages the intra-series sparse graph constructed in Section 3.2 to capture the mutual dependencies between patches. In this section, we use the time node embedding $E^t \in \mathbb{R}^{N\times P\times d}$, where $P$ represents the number of patches, and $d$ denotes the feature dimension of the time nodes. To reduce computational complexity, we apply average pooling over the variable dimension $N$, resulting in: $E^t \in \mathbb{R}^{N\times P\times d} \to \mathbb{R}^{P\times d}$. Then, by introducing a graph convolutional network, the model can effectively learn the dynamic relationships between different time steps along the time dimension. The formula is as follows:

$$E_G^t = \sigma(\hat{\mathrm{A}}^t E^t W^t), \tag{11}$$

$$\hat{A}^t = D^{(-\frac{1}{2})}(A^t+I)D^{(\frac{1}{2})}, \tag{12}$$

where $E_G^t \in \mathbb{R}^{P\times d}$ denotes the patch interaction features, and $\hat{A}^t \in \mathbb{R}^{P\times P}$ denotes the normalized adjacency matrix with self-loops. The superscript $t$ represents the time flow. Here, $I$ represents the identity matrix, $D$ represents the degree matrix, $\sigma$ is the activation function, and $W^t$ is the learnable parameter matrix. Through multilayer Patch-GCN, the model can progressively learn the dynamic dependencies between time steps and capture features such as periodicity in the data. Then, we apply a residual connection: $E_G^t = E_G^t + E^t \to \mathbb{R}^{N\times P\times d}$, which helps prevent potential information loss during the average pooling process.

In this chapter, we use GCN to explicitly learn dependencies from two dimensions for multivariate time series data. First, based on the intra-series and inter-series dynamic sparse graphs constructed in Section 3.2, we design multilayer Variable-GCN and multilayer Patch-GCN to effectively learn the spatial dependencies between variables and the dynamic dependencies between patches. However, this approach still primarily focuses on modeling local dependencies, and there may still be limitations in learning global dependencies, particularly the implicit dependencies between variables and patches.

### 3.4. Dual-flow global attention layer

In this section, we introduce the global attention mechanism, which aims to further improve the expressive power of the model by learning the global dependencies between and within series.

#### 3.4.1. Inter-series global attention module

In the Inter-series global attention layer, our goal is to further capture the global dependencies between different variables using a global attention mechanism. In the variable flow, although Variable-GCNs effectively learn explicit local dependencies between variables, there may exist latent implicit dependencies between variables in multivariate time series data, especially when complex interactions occur between different variables. Motivated by this, we adopt a global attention mechanism to weight and aggregate features by calculating the global correlations between variable nodes. Specifically, $E_G^v = \{E_{G,1}^v, E_{G,2}^v, \dots, E_{G,N}^v\}$ is updated as $E_A^v = \{E_{A,1}^v, E_{A,2}^v, \dots, E_{A,N}^v\}$ by integrating global variable information. The subscript $A$ represents the result of the attention operation. The formula is as follows:

$$E_{A,i}^v = \sum_{j=1}^{N} \alpha_{ij}(W_V^v E_{G,i}^v), \tag{13}$$

where $E_{A,i}^v$ denotes the updated feature for the $i$th variable, and the relation coefficient $\alpha_{ij}$ determines the amount of information each feature receives from other variables. It is computed as,

$$\alpha_{ij} = \frac{\exp(\varphi_{ij})}{\sum_{j=1}^{N}\exp(\varphi_{ij})}, \tag{14}$$

here, $\varphi_{ij}$ is the scaled attention score, defined as:

$$\varphi_{ij} = \frac{(W_Q^v E_{G,i}^v)(W_K^v E_{G,j}^v)^T}{\sqrt{d}}, \tag{15}$$

where $W_Q^v$, $W_K^v$, and $W_V^v$ are the learnable parameters.

#### 3.4.2. Intra-series global attention module

In the Intra-series global attention layer, our goal is to capture the global dependencies between time steps. Although the multi-layer Patch-GCN can progressively aggregate local features between patches, this local dependency modeling may be insufficient for capturing the global information across the entire time series. Therefore, we introduce an Intra-series global attention mechanism that weights and aggregates information by calculating the correlations between each patch. Specifically, $E_G^t = \{E_{G,1}^t, E_{G,2}^t, \dots, E_{G,P}^t\}$ is updated as $E_A^t = \{E_{A,1}^t, E_{A,2}^t, \dots, E_{A,P}^t\}$ by integrating the global temporal information. The formula is as follows:

$$E_{A,i}^t = \sum_{j=1}^{P} \alpha_{ij}(W_V^t E_{G,i}^t), \tag{16}$$

$$\alpha_{ij} = \frac{\exp(\varphi_{ij})}{\sum_{j=1}^{P}\exp(\varphi_{ij})}, \tag{17}$$

$$\varphi_{ij} = \frac{(W_Q^t E_{G,i}^t)(W_K^t E_{G,j}^t)^T}{\sqrt{d}}, \tag{18}$$

where $E_{A,i}^t$ represents the updated feature for the $i$th Patch, and the relation coefficient $\alpha_{ij}$ determines the amount of information each feature receives from other Patches. $\varphi_{ij}$ is the scaled attention score. $W_Q^t$, $W_K^t$, and $W_V^t$ are the learnable parameters.

### 3.5. Dual-flow fusion module

In this section, we explore how to effectively integrate the features from the time flow and variable flow. Specifically, we adopt a strategy that combines concatenation and linear projection to merge these two types of information, where the features from the time flow are represented as $E_A^t \in \mathbb{R}^{N\times P\times d}$, and the features from the variable flow are represented as $E_A^v \in \mathbb{R}^{N\times d}$. First, to ensure consistency in dimensions between the time flow and variable flow, we process the time flow features. Similar to the method used in PatchTST [26], we apply flattening and linear transformation to convert the time flow dimensions into a form that matches those of the variable flow: $E_A^t \in \mathbb{R}^{N\times P\times d} \to \mathbb{R}^{N\times d}$.

Next, through the concatenation operation, we concatenate the feature vectors from the time flow and variable flow along the last dimension to obtain the new feature representation $E_{cat}$, as shown in the following formula:

$$E_{cat} = \mathrm{Concat}(E_A^t, E_A^v) \in \mathbb{R}^{N\times 2d}. \tag{19}$$

Then, a linear projection layer is introduced, where a trainable linear mapping matrix $W^{out} \in \mathbb{R}^{2d\times\eta}$ is used to map the concatenated features to the output space, resulting in the final prediction $E_{out}$:

$$E_{out} = E_{cat}W^{out} \in \mathbb{R}^{N\times\eta}, \tag{20}$$

where $\eta$ is the length of the output prediction. Through this projection layer, the model is able to learn the weights across different dimensions, enabling the deep fusion of time flow and variable flow features. This fusion approach not only preserves the feature information of both time flow and variable flow but also allows for a more expressive integrated feature through linear transformation, thereby effectively enhancing the model's performance.

### 3.6. Complexity analysis

In this section, we analyze the time and space complexity of each component in DFGCN. We categorize the components into two primary flows: the time flow, which models intra-series dependencies, and the variable flow, which captures inter-series correlations.

As shown in Table 1, we analyze time and space complexity across key components, including node embeddings, dynamic sparse graphs, multilayer GCN, and global attention mechanisms. Time complexity analysis: The time complexity of node embeddings is $\mathcal{O}(PSd)$ for time flow and $\mathcal{O}(NLd)$ for variable flow. Constructing intra-series and inter-series dynamic sparse graphs involves pairwise correlation computations and top-k selection, resulting in complexities of $\mathcal{O}\left(P^2S + P^2\log P\right)$ and $\mathcal{O}\left(N^2L + N^2\log N\right)$, respectively. Multilayer GCN aggregates node features with complexities represented by $\mathcal{O}\left(Pkd + Pd^2\right)$ and $\mathcal{O}\left(Nkd + Nd^2\right)$. Global attention mechanisms introduce additional quadratic terms, $\mathcal{O}\left(P^2d + Pd^2\right)$ for intra-series attention and $\mathcal{O}\left(N^2d + Nd^2\right)$ for inter-series attention, making them

**Table 1**
Analysis of the time and space complexity of DFGCN. The parameters are defined as follows: $N$ represents the number of variables, $L$ denotes the input length, $d$ is the model dimension, $k$ indicates the number of adjacent edges per node, $P$ represents the number of patch subsequences, and $S$ refers to the patch length.

| Components | | Time complexity | Space complexity |
|---|---|---|---|
| Time flow | Time node embeddings | $\mathcal{O}(PSd)$ | $\mathcal{O}(Sd)$ |
| | Intra-series dynamic sparse graph | $\mathcal{O}\left(P^2S + P^2 \log P\right)$ | $\mathcal{O}\left(P^2 + PS\right)$ |
| | Multilayer Patch-GCN | $\mathcal{O}\left(Pkd + Pd^2\right)$ | $\mathcal{O}\left(Pd + d^2\right)$ |
| | Intra-series global attention | $\mathcal{O}\left(P^2d + Pd^2\right)$ | $\mathcal{O}\left(P^2 + Pd\right)$ |
| Variable flow | Variable node embeddings | $\mathcal{O}(NLd)$ | $\mathcal{O}(Ld)$ |
| | Inter-series dynamic sparse graph | $\mathcal{O}\left(N^2L + N^2 \log N\right)$ | $\mathcal{O}\left(N^2 + NL\right)$ |
| | Multilayer variable-GCN | $\mathcal{O}\left(Nkd + Nd^2\right)$ | $\mathcal{O}\left(Nd + d^2\right)$ |
| | Inter-series global attention | $\mathcal{O}\left(N^2d + Nd^2\right)$ | $\mathcal{O}\left(N^2 + Nd\right)$ |

**Table 2**
Statistics of 11 MTSF benchmark datasets from different fields.

| Datasets | Information | Prediction length | Train/Valid/Test | Timesteps | Number of variates | Frequency |
|---|---|---|---|---|---|---|
| ETTm1/ETTm2 | Temperature | {96,192,336,720} | 6:2:2 | 69 680 | 7 | 15 mins |
| ETTh1/ETTh2 | Temperature | {96,192,336,720} | 6:2:2 | 17 420 | 7 | 1 h |
| Flight | Flight | {96,192,336,720} | 7:1:2 | 26 211 | 7 | 1 h |
| Exchange | Economy | {96,192,336,720} | 7:1:2 | 7588 | 8 | 1 day |
| Weather | Weather | {96,192,336,720} | 7:1:2 | 52 696 | 21 | 10 mins |
| Solar | Energy | {96,192,336,720} | 7:1:2 | 52 179 | 137 | 10 mins |
| PEMS08 | Transportation | {12,24,48,96} | 6:2:2 | 17 786 | 170 | 5 mins |
| Electricity | Electricity | {96,192,336,720} | 7:1:2 | 26 304 | 321 | 1 h |
| Traffic | Transportation | {96,192,336,720} | 7:1:2 | 17 544 | 862 | 1 h |

major computational bottlenecks. Space complexity analysis: The memory requirements include $\mathcal{O}(Sd)$ and $\mathcal{O}(Ld)$ for storing time and variable embeddings. The adjacency matrices for intra-series and inter-series graphs require $\mathcal{O}\left(P^2 + PS\right)$ and $\mathcal{O}\left(N^2 + NL\right)$ space. Multilayer GCN demand $\mathcal{O}\left(Pd + d^2\right)$ and $\mathcal{O}\left(Nd + d^2\right)$, while global attention mechanisms further increase memory consumption to $\mathcal{O}\left(P^2 + Pd\right)$ and $\mathcal{O}\left(N^2 + Nd\right)$. By incorporating this complexity analysis, we provide a more comprehensive evaluation of DFGCN's computational costs, ensuring clarity and transparency in our model's computational analysis. This analysis also serves as a foundation for potential optimizations in future work.

## 4. Experiments

We evaluate the proposed DFGCN model on multiple public time series datasets to validate the generalization capability of the framework. Additionally, further in-depth analysis reveals the effectiveness of the method. The experimental design also delves into the effectiveness of the model components.

### 4.1. Datasets and baselines

***Datasets.*** To validate the effectiveness and generalization capability of the proposed DFGCN model, we select 11 widely used public multivariate time series datasets that cover multiple domains, ensuring the model's applicability across various real-world scenarios. Table 2 presents detailed information about these datasets, including the **ETT** dataset (electric transformer temperature data with four subsets sampled at 15 min and 1-h intervals), the **Electricity** dataset (hourly electricity usage data from 321 customers), the **Weather** dataset (21 meteorological indicators from weather stations in Germany), the **Traffic** dataset (road occupancy data from highways in San Francisco), the **Solar-Energy** dataset (solar energy data from 137 photovoltaic plants), the **Exchange-rate** dataset (exchange rate data from 8 countries), the **PEMS08** dataset (traffic flow data from California's public transportation network), and the **Flight** dataset (flight information from the OpenSky dataset). These datasets provide diverse time series data for evaluating the DFGCN model, helping to verify its cross-domain generalization ability.

***Baselines.*** To comprehensively evaluate the proposed DFGCN model, we select several representative baseline models that cover various architectures, including Transformer, Graph Neural Networks, Convolutional Neural Networks, and Multilayer Perceptrons. Below is a brief introduction to each type of baseline model. *Transformer-based Models*: (1) **iTransformer** [23]: Embeds each variable as a token and uses attention mechanisms to capture correlations in multivariate time series, showcasing strong modeling abilities. (2) **BasisFormer** [21]: Leverages adaptive self-supervised learning for trends and seasonality, and a bidirectional cross-attention mechanism to calculate similarity between time series, trends, and seasonality, offering high interpretability and flexibility. (3) **PatchTST** [26]: A Transformer-based model that splits time series into subsequence patches, improving long-term forecasting accuracy and demonstrating excellent self-supervised learning. (4) **Leddam** [28]: A trainable trend-seasonality decomposition module is proposed, which enhances time series decomposition by utilizing learnable convolutional kernels initialized with a Gaussian distribution. And a dual-attention module is designed to efficiently capture both inter-series dependencies and intra-series dynamic variations. *GNN-based Models*: (5) **CrossGNN** [47]: By introducing adaptive multi-scale identifiers and a cross-scale, cross-variable graph neural network, the model refines the handling of noise in multivariate time series data while maintaining linear time and space complexity, making it particularly effective for processing noisy time series. (6) **MSGNet** [12]: By leveraging frequency-domain analysis and adaptive graph convolution, the model captures cross-series correlations across multiple temporal scales. Additionally, it incorporates a self-attention mechanism to model intra-series dependencies, thereby enhancing the model's generalization ability and interpretability. (7) **FourierGNN** [36]: Redefines MTSF by using hyper-variable graphs and fully connected spatiotemporal graphs, unifying spatiotemporal dependency modeling and enhancing performance. (8) **DyGraphformer** [29]: Dynamic graph convolutional layers are integrated into the Transformer to model dynamic spatial dependencies among variables, while segmented embedding of time series is employed to reduce computational complexity. *CNN-based Model*: (9) **TimesNet** [18]: A time series analysis model based on TimesBlock that converts 1D time series into 2D tensors, adaptively discovering multiple periodicities and using efficient convolution modules to capture complex time series variations, addressing the modeling of diverse time dynamics. *CNN-based Model*: (10) **DLinear** [27]: Decomposes time series into trend and residual components, using a single

linear layer to model the time dependencies of each part, providing a simple and effective forecasting approach.

### 4.2. Implementation details

To ensure a fair comparison with existing work, all experiments followed the previous settings. Specifically, we set the look-back window size to 96, which is consistent with all baseline models. Like iTransformer model, for the PEMS08 dataset, the prediction lengths are set to {12, 24, 36, 48}, while for other datasets, the prediction lengths are set to {96, 192, 336, 720}, which is consistent with previous work settings [12,18,47]. All experiments are implemented on the PyTorch 1.8 platform, and model training is conducted using NVIDIA RTX 4090D 24 GB and RTX 3060 12 GB GPUs. Consistent with PatchTST [26], we use the Adam optimizer and employ L1 and L2 loss functions for model optimization to better ensure model convergence performance. The maximum number of training epochs is set to 10, and training will be stopped early if the validation loss does not decrease for 3 consecutive epochs, to prevent overfitting. We set Top-K to 2 for all datasets. The hyperparameters of the model, including Patch length, model stacking layers, learning rate, batch size, and model dimensions, are specified for each dataset and shown in Table 5. furthermore, for the baseline model experiments, all results are obtained from the source manuscripts of the baseline models. For datasets not provided in the manuscripts, we implement the models using the official code provided by the baselines. Through these unified experimental settings, we ensure a fair comparison with existing methods, allowing for a more effective evaluation of the performance of the proposed DFGCN model.

***Evaluation metrics.*** To maintain consistency with all baseline models, we use Mean Squared Error (MSE) and Mean Absolute Error (MAE) as evaluation metrics to assess the predictive performance of the model. MSE measures the average squared difference between predicted and actual values, reflecting prediction accuracy and the impact of large errors. MAE calculates the average absolute difference between predicted and actual values, providing an intuitive measure of prediction error. These two metrics comprehensively evaluate the model's performance.

### 4.3. Main results

The evaluation results of the DFGCN model and other baseline models on 11 datasets are shown in Table 6. Generally, DFGCN achieves the best results on most datasets, with improvements ranging from 2.40% to 19.84%. Additionally, out of all 88 experimental results, DFGCN ranks first in 54 cases. A detailed analysis reveals the following: (1) Compared to Transformer-based models, DFGCN achieves a 2.40% to 7.35% improvement. iTransformer focuses on learning variable dependencies, while PatchTST and BasisFormer emphasize learning in the temporal dimension. Leddam introduces an innovative learnable trend-seasonality decomposition module coupled with a dual-level attention mechanism to simultaneously capture both inter-series dependencies and intra-series variations. In comparison to these models, the proposed DFGCN exhibits a notable advantage by jointly capturing cross-variable and cross-temporal dependencies, thereby enabling comprehensive modeling of spatiotemporal features and effectively mitigating the challenges posed by dependency heterogeneity. (2) When compared with GNN-based models, DFGCN achieves a 4.88% to 11.95% improvement. While models like DyGraphformer, MSGNet, CrossGNN, and FourierGNN address both inter-series and intra-series dependencies in an integrated fashion, the strength of the DFGCN model lies in its dual-flow architecture, which separately models inter-series and intra-series interactions. This design effectively reduces the mutual interference between the two types of dependency learning, which frequently occurs in single-flow architectures. (3) Compared to CNN-based model, DFGCN achieves a 13.44% performance improvement over the TimesNet model. While TimesNet aims to capture multi-periodic variations, it primarily focuses on modeling the temporal dimension and overlooks inter-series dependencies. (4) Compared to MLP-based model, DFGCN outperforms the DLinear model with a 19.84% performance improvement. DLinear primarily learns trends and residuals in time series but lacks deep learning of intra-series periodicity and inter-series dependencies, resulting in relatively weaker performance. Overall, DFGCN demonstrates superior performance across various baseline models, effectively capturing both inter-series and intra-series latent features, thereby significantly enhancing MTSF tasks.

### 4.4. Component ablation study

To validate the effectiveness of the proposed method, we evaluate the contributions of different components of the model. The ablation study focuses on addressing four main aspects: (1) effectiveness of time flow; (2) effectiveness of variable flow; (3) effectiveness of GCN (explicit local dependencies); and (4) effectiveness of attention (global dependencies).

***Effectiveness of time flow***. To evaluate the impact of the time flow on prediction performance, we remove the time flow component, and the results are shown in Table 3. The results indicate that removing the time flow module leads to a decline in model performance across all datasets. Notably, significant performance drops are observed on the Solar and ETTm1 datasets, with decreases of 2.52% and 2.32%, respectively. This demonstrates that the time flow component contributes to improving the model's predictive performance and suggests that it effectively captures temporal dependencies in multivariate time series.

***Effectiveness of variable flow***. To evaluate the impact of the variable flow on prediction performance, we remove the variable flow component, and the results are shown in Table 3. The results clearly show that the removal of the variable flow component leads to a decline in prediction performance across all datasets, with decreases ranging from 0.51% to 13.03%. Especially on the Electricity, Solar, and Traffic datasets, which have larger variable dimensions, the performance decreased by 9.68%, 13.03%, and 8.01%, respectively. These findings demonstrate that cross-variable learning is critical for handling high-dimensional datasets. The variable flow component plays a key role in modeling inter-variable dependencies, thereby significantly enhancing the model's predictive capabilities.

***Effectiveness of GCN***. To evaluate the contribution of the GCN component to the model's performance, we remove the GCN component, and the experimental results are shown in Table 4. As observed, the prediction performance consistently declines across nine benchmark datasets. Notably, significant performance degradation occurs on the Electricity, Traffic, and Solar datasets. This indicates that the GCN component plays a crucial role in explicitly modeling local dependencies.

***Effectiveness of attention.*** As shown in Table 4, removing the Attention component results in a significant decline in prediction performance across all benchmark datasets. This demonstrates that the Attention component effectively captures global inter-series dependencies. Further analysis reveals that the performance drop is more pronounced for datasets with higher variable dimensions, further validating the importance of modeling inter-variable dependencies in multivariate datasets.

### 4.5. Parameter sensitivity analysis

#### 4.5.1. Sensitivity analysis of Top-K

As shown in Fig. 3, we conduct a sensitivity analysis of the Top-K parameter on the Electricity, Traffic, Solar, and Weather datasets to understand its impact on model performance. Top-K refers to the selection of the top K most relevant variable nodes based on similarity weights, which are used to construct the adjacency matrix for the GCN. In the experiments, due to the relatively small number of patches, we

**Table 3**
Ablation study on the contribution of time flow and variable flow to model performance. w/o means without.

| Models | | ETTh2 | | ETTh1 | | ETTm1 | | ETTm2 | | Exchange | | Weather | | Electricity | | Solar | | Traffic | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Metric | | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE |
| DFGCN | 96 | 0.281 | 0.333 | 0.371 | 0.389 | 0.319 | 0.352 | 0.172 | 0.255 | 0.082 | 0.201 | 0.156 | 0.198 | 0.146 | 0.236 | 0.189 | 0.220 | 0.409 | 0.270 |
| | 192 | 0.362 | 0.387 | 0.421 | 0.416 | 0.363 | 0.373 | 0.239 | 0.298 | 0.174 | 0.296 | 0.207 | 0.246 | 0.161 | 0.249 | 0.226 | 0.245 | 0.431 | 0.277 |
| | 336 | 0.409 | 0.424 | 0.462 | 0.438 | 0.399 | 0.397 | 0.302 | 0.339 | 0.329 | 0.414 | 0.265 | 0.289 | 0.177 | 0.266 | 0.246 | 0.262 | 0.446 | 0.283 |
| | 720 | 0.427 | 0.443 | 0.460 | 0.454 | 0.467 | 0.435 | 0.400 | 0.397 | 0.832 | 0.686 | 0.344 | 0.341 | 0.210 | 0.294 | 0.251 | 0.265 | 0.479 | 0.301 |
| | avg | 0.383 | | 0.426 | | 0.388 | | 0.300 | | 0.377 | | 0.256 | | 0.217 | | 0.238 | | 0.362 | |
| w/o Time Flow | 96 | 0.289 | 0.339 | 0.376 | 0.392 | 0.333 | 0.361 | 0.176 | 0.257 | 0.085 | 0.205 | 0.164 | 0.204 | 0.149 | 0.240 | 0.196 | 0.223 | 0.422 | 0.278 |
| | 192 | 0.376 | 0.393 | 0.426 | 0.421 | 0.379 | 0.382 | 0.239 | 0.299 | 0.178 | 0.300 | 0.213 | 0.249 | 0.165 | 0.253 | 0.232 | 0.250 | 0.438 | 0.283 |
| | 336 | 0.421 | 0.430 | 0.466 | 0.441 | 0.409 | 0.400 | 0.305 | 0.340 | 0.327 | 0.414 | 0.270 | 0.291 | 0.182 | 0.270 | 0.254 | 0.266 | 0.454 | 0.290 |
| | 720 | 0.434 | 0.447 | 0.468 | 0.462 | 0.476 | 0.437 | 0.406 | 0.398 | 0.850 | 0.695 | 0.351 | 0.344 | 0.219 | 0.301 | 0.261 | 0.270 | 0.485 | 0.308 |
| | Decline | 0.391(↓2.08%) | | 0.432(↓1.41%) | | 0.397(↓2.32%) | | 0.303(↓1.00%) | | 0.382(↓1.33%) | | 0.261(↓1.95%) | | 0.222(↓2.30%) | | 0.244(↓2.52%) | | 0.370(↓2.21%) | |
| w/o Variable Flow | 96 | 0.283 | 0.335 | 0.380 | 0.393 | 0.327 | 0.356 | 0.175 | 0.258 | 0.086 | 0.206 | 0.168 | 0.211 | 0.175 | 0.256 | 0.219 | 0.251 | 0.451 | 0.299 |
| | 192 | 0.368 | 0.390 | 0.430 | 0.422 | 0.367 | 0.376 | 0.241 | 0.301 | 0.178 | 0.301 | 0.214 | 0.251 | 0.183 | 0.264 | 0.261 | 0.275 | 0.463 | 0.298 |
| | 336 | 0.413 | 0.426 | 0.472 | 0.442 | 0.401 | 0.399 | 0.303 | 0.341 | 0.334 | 0.419 | 0.270 | 0.293 | 0.199 | 0.280 | 0.282 | 0.288 | 0.478 | 0.304 |
| | 720 | 0.431 | 0.446 | 0.482 | 0.469 | 0.463 | 0.433 | 0.401 | 0.398 | 0.846 | 0.694 | 0.347 | 0.342 | 0.237 | 0.311 | 0.290 | 0.289 | 0.510 | 0.323 |
| | Decline | 0.387(↓1.04%) | | 0.436(↓2.35%) | | 0.390(↓0.51%) | | 0.302(↓0.67%) | | 0.383(↓1.59%) | | 0.262(↓2.34%) | | 0.238(↓9.68%) | | 0.269(↓13.03%) | | 0.391(↓8.01%) | |

**Table 4**
Ablation study on the contribution of GCN module and attention module to model performance.

| Models | | ETTh2 | | ETTh1 | | ETTm1 | | ETTm2 | | Exchange | | Weather | | Electricity | | Solar | | Traffic | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Metric | | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE |
| DFGCN | 96 | 0.281 | 0.333 | 0.371 | 0.389 | 0.319 | 0.352 | 0.172 | 0.255 | 0.082 | 0.201 | 0.156 | 0.198 | 0.146 | 0.236 | 0.189 | 0.220 | 0.409 | 0.270 |
| | 192 | 0.362 | 0.387 | 0.421 | 0.416 | 0.363 | 0.373 | 0.239 | 0.298 | 0.174 | 0.296 | 0.207 | 0.246 | 0.161 | 0.249 | 0.226 | 0.245 | 0.431 | 0.277 |
| | 336 | 0.409 | 0.424 | 0.462 | 0.438 | 0.399 | 0.397 | 0.302 | 0.339 | 0.329 | 0.414 | 0.265 | 0.289 | 0.177 | 0.266 | 0.246 | 0.262 | 0.446 | 0.283 |
| | 720 | 0.427 | 0.443 | 0.460 | 0.454 | 0.467 | 0.435 | 0.400 | 0.397 | 0.832 | 0.686 | 0.344 | 0.341 | 0.210 | 0.294 | 0.251 | 0.265 | 0.479 | 0.301 |
| | avg | 0.383 | | 0.426 | | 0.388 | | 0.300 | | 0.377 | | 0.256 | | 0.217 | | 0.238 | | 0.362 | |
| w/o GCN | 96 | 0.289 | 0.336 | 0.379 | 0.393 | 0.327 | 0.361 | 0.175 | 0.257 | 0.083 | 0.202 | 0.168 | 0.209 | 0.154 | 0.244 | 0.201 | 0.239 | 0.438 | 0.290 |
| | 192 | 0.363 | 0.388 | 0.434 | 0.425 | 0.371 | 0.380 | 0.245 | 0.303 | 0.172 | 0.294 | 0.213 | 0.250 | 0.167 | 0.254 | 0.242 | 0.260 | 0.453 | 0.292 |
| | 336 | 0.415 | 0.424 | 0.475 | 0.443 | 0.401 | 0.400 | 0.305 | 0.342 | 0.332 | 0.416 | 0.270 | 0.290 | 0.184 | 0.272 | 0.261 | 0.272 | 0.472 | 0.299 |
| | 720 | 0.429 | 0.445 | 0.495 | 0.478 | 0.466 | 0.437 | 0.403 | 0.400 | 0.840 | 0.689 | 0.349 | 0.343 | 0.224 | 0.304 | 0.268 | 0.273 | 0.502 | 0.318 |
| | Decline | 0.386(↓0.78%) | | 0.440(↓3.28%) | | 0.393(↓1.29%) | | 0.304(↓1.33%) | | 0.379(↓0.53%) | | 0.262(↓2.34%) | | 0.225(↓3.69%) | | 0.252(↓5.88%) | | 0.383(↓5.80%) | |
| w/o Attention | 96 | 0.287 | 0.335 | 0.381 | 0.388 | 0.341 | 0.362 | 0.180 | 0.260 | 0.082 | 0.200 | 0.183 | 0.220 | 0.171 | 0.253 | 0.229 | 0.245 | 0.471 | 0.295 |
| | 192 | 0.375 | 0.391 | 0.434 | 0.418 | 0.382 | 0.380 | 0.242 | 0.299 | 0.175 | 0.297 | 0.230 | 0.260 | 0.180 | 0.262 | 0.269 | 0.267 | 0.477 | 0.296 |
| | 336 | 0.417 | 0.427 | 0.475 | 0.438 | 0.416 | 0.401 | 0.307 | 0.340 | 0.327 | 0.412 | 0.282 | 0.296 | 0.195 | 0.277 | 0.297 | 0.283 | 0.488 | 0.301 |
| | 720 | 0.435 | 0.447 | 0.478 | 0.465 | 0.476 | 0.435 | 0.405 | 0.395 | 0.840 | 0.689 | 0.355 | 0.344 | 0.235 | 0.310 | 0.296 | 0.285 | 0.521 | 0.317 |
| | Decline | 0.389(↓1.56%) | | 0.435(↓2.11%) | | 0.399(↓2.83%) | | 0.304(↓1.33%) | | 0.378(↓0.26%) | | 0.271(↓5.86%) | | 0.235(↓8.29%) | | 0.271(↓13.86%) | | 0.396(↓9.39%) | |

**Table 5**
Detailed hyperparameter settings for eleven datasets.

| Datasets | Flight | Electricity | Weather | ETTh1 | ETTh2 | ETTm1 | ETTm2 | Solar | Exchange | Traffic | PEMS08 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| # d_model | 256 | 512 | 512 | 128 | 64 | 128 | 64 | 512 | 128 | 512 | 512 |
| # batch_size | 32 | 16 | 8 | 32 | 64 | 8 | 32 | 16 | 32 | 16 | 32 |
| # learning_rate | 0.001 | 0.001 | 0.001 | 0.001 | 0.001 | 0.001 | 0.001 | 0.001 | 0.001 | 0.001 | 0.001 |
| # e_layers | 1 | 2 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 2 | 1 |
| # patch_len | 16 | 24 | 16 | 24 | 16 | 16 | 16 | 16 | 16 | 24 | 16 |

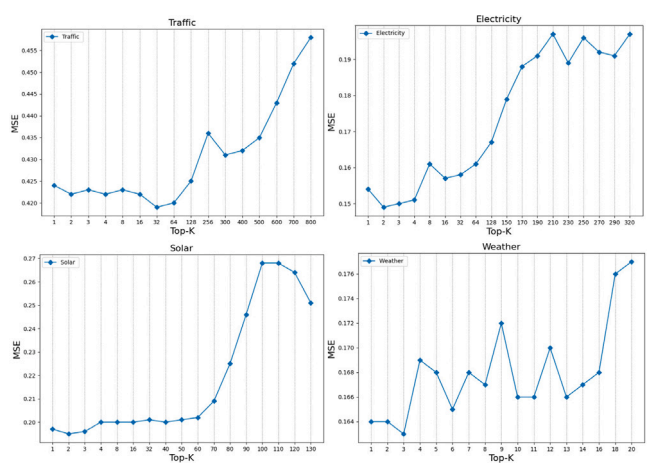


**Fig. 3.** The impact of Top-K on the prediction results. Results of Traffic, Electricity, Solar and Weather at different Top-K (with input-96-predict-96 configuration).

could not fully explore the impact of Top-K on model performance. Therefore, we focus our analysis on the variable flow to more clearly demonstrate the effect of Top-K size on model performance. The results show that as Top-K increases, model performance significantly declines. This is because a larger Top-K increases the number of neighboring nodes in the graph, which introduces more variables that may include noise irrelevant to the target variable. This noise can interfere with the model's learning and reduce its predictive performance. To ensure stability and superiority across all datasets, we set Top-K to 2 in this paper.

### 4.5.2. Sensitivity analysis of patch length

This experiment explores the impact of patch length on prediction performance. We validate the model on the ETTh1, ETTh2, Weather, and Traffic datasets by adjusting the patch length $P$ to $[4, 8, 12, 16, 20, 24, 28]$. The step size is set to match the patch length to ensure no overlap between patches. As shown in Fig. 4, the MSE score changes with different choices of $P$, indicating that the DFGCN model is sensitive to patch length. Further analysis reveals that variations in $P$ alter the internal graph structure of the sequence, thus affecting the model's performance. The optimal patch length may depend on the dataset, but $P = [16, 24]$ generally produces better results, as shown in Table 5.

### 4.5.3. Sensitivity analysis of learning rate

The learning rate is an important hyperparameter during the training of the model. Consistent with PatchTST and iTransformer, when training the DFGCN model, only the initial learning rate needs to be set, and it is halved after each epoch. To evaluate the impact of the learning rate on model performance, we conducted experiments on six datasets, with the results shown in Fig. 5. It can be observed that

**Table 6**
Multivariate series forecasting results on eleven datasets from different fields. A lower MSE and MAE indicates better performance, and the best results are highlighted in bold. Avg represents the average value of the experimental results.

| Models | | DFGCN (ours) | | Transformer-based | | | | | | | | GNN-based | | | | | | | | CNN-based | | MLP-based | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | Leddam (2024) | | iTranformer (2024) | | BasisFormer (2024) | | PatchTST (2023) | | DyGraphformer (2025) | | MSGNet (2024) | | CrossGNN (2023) | | FourierGNN (2024) | | TimesNet (2023) | | DLinear (2023) | |
| Metric | | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE |
| ETTh2 | 96 | **0.281** | **0.333** | 0.292 | 0.343 | 0.297 | 0.349 | 0.312 | 0.356 | 0.294 | 0.343 | 0.303 | 0.357 | 0.328 | 0.371 | 0.309 | 0.359 | 0.305 | 0.352 | 0.340 | 0.374 | 0.333 | 0.387 |
| | 192 | **0.362** | **0.387** | 0.367 | 0.389 | 0.380 | 0.400 | 0.382 | 0.401 | 0.377 | 0.393 | 0.379 | 0.402 | 0.402 | 0.414 | 0.390 | 0.406 | 0.380 | 0.399 | 0.402 | 0.414 | 0.477 | 0.476 |
| | 336 | 0.409 | 0.424 | 0.412 | 0.424 | 0.428 | 0.432 | 0.418 | 0.431 | **0.381** | **0.409** | 0.437 | 0.441 | 0.435 | 0.443 | 0.426 | 0.444 | 0.426 | 0.438 | 0.452 | 0.452 | 0.594 | 0.541 |
| | 720 | 0.427 | 0.443 | 0.419 | 0.438 | 0.427 | 0.445 | 0.418 | 0.438 | **0.412** | **0.433** | 0.444 | 0.449 | 0.417 | 0.441 | 0.445 | 0.464 | 0.448 | 0.456 | 0.462 | 0.468 | 0.831 | 0.657 |
| ETTh1 | 96 | **0.371** | **0.389** | 0.377 | 0.394 | 0.386 | 0.405 | 0.394 | 0.411 | 0.393 | 0.408 | 0.380 | 0.404 | 0.390 | 0.411 | 0.382 | 0.398 | 0.396 | 0.407 | 0.384 | 0.402 | 0.386 | 0.400 |
| | 192 | **0.421** | **0.416** | 0.424 | 0.422 | 0.441 | 0.436 | 0.442 | 0.437 | 0.445 | 0.434 | 0.429 | 0.437 | 0.442 | 0.442 | 0.427 | 0.425 | 0.448 | 0.435 | 0.436 | 0.429 | 0.437 | 0.432 |
| | 336 | 0.462 | **0.438** | **0.459** | 0.442 | 0.487 | 0.458 | 0.473 | 0.451 | 0.484 | 0.451 | 0.472 | 0.453 | 0.480 | 0.468 | 0.465 | 0.445 | 0.493 | 0.457 | 0.491 | 0.469 | 0.481 | 0.459 |
| | 720 | **0.460** | **0.454** | 0.463 | 0.459 | 0.503 | 0.491 | **0.460** | 0.465 | 0.480 | 0.471 | 0.493 | 0.479 | 0.494 | 0.488 | 0.472 | 0.468 | 0.483 | 0.471 | 0.521 | 0.500 | 0.519 | 0.516 |
| ETTm1 | 96 | **0.319** | **0.352** | 0.319 | 0.359 | 0.334 | 0.368 | 0.342 | 0.374 | 0.321 | 0.360 | 0.323 | 0.364 | **0.319** | 0.366 | 0.335 | 0.373 | 0.326 | 0.358 | 0.338 | 0.375 | 0.345 | 0.372 |
| | 192 | 0.363 | **0.373** | 0.369 | 0.383 | 0.377 | 0.391 | 0.380 | 0.392 | **0.362** | 0.384 | 0.369 | 0.390 | 0.376 | 0.397 | 0.372 | 0.390 | 0.373 | 0.384 | 0.374 | 0.387 | 0.380 | 0.389 |
| | 336 | 0.399 | **0.397** | 0.394 | 0.402 | 0.426 | 0.420 | 0.420 | 0.418 | **0.392** | 0.402 | 0.413 | 0.420 | 0.417 | 0.422 | 0.403 | 0.411 | 0.413 | 0.410 | 0.410 | 0.411 | 0.413 | 0.413 |
| | 720 | 0.467 | 0.436 | 0.460 | 0.442 | 0.491 | 0.459 | 0.492 | 0.458 | **0.450** | **0.435** | 0.459 | 0.453 | 0.481 | 0.458 | 0.461 | 0.442 | 0.488 | 0.451 | 0.478 | 0.450 | 0.474 | 0.453 |
| ETTm2 | 96 | **0.172** | **0.255** | 0.176 | 0.257 | 0.180 | 0.264 | 0.184 | 0.266 | 0.178 | 0.260 | 0.177 | 0.260 | 0.177 | 0.262 | 0.176 | 0.266 | 0.177 | 0.257 | 0.187 | 0.267 | 0.193 | 0.292 |
| | 192 | **0.239** | **0.298** | 0.243 | 0.303 | 0.250 | 0.309 | 0.248 | 0.307 | 0.249 | 0.307 | 0.247 | 0.308 | 0.247 | 0.307 | 0.240 | 0.307 | 0.244 | 0.301 | 0.249 | 0.309 | 0.284 | 0.362 |
| | 336 | **0.302** | **0.339** | 0.303 | 0.341 | 0.311 | 0.348 | 0.321 | 0.355 | 0.313 | 0.346 | 0.306 | 0.349 | 0.312 | 0.346 | 0.304 | 0.345 | 0.303 | 0.340 | 0.321 | 0.351 | 0.369 | 0.427 |
| | 720 | **0.400** | **0.397** | **0.400** | 0.398 | 0.412 | 0.407 | 0.410 | 0.404 | **0.400** | 0.398 | 0.424 | 0.414 | 0.414 | 0.403 | 0.406 | 0.400 | 0.401 | 0.398 | 0.408 | 0.403 | 0.554 | 0.522 |
| Weather | 96 | **0.156** | **0.198** | 0.156 | 0.202 | 0.174 | 0.214 | 0.173 | 0.214 | 0.178 | 0.219 | 0.157 | 0.205 | 0.163 | 0.212 | 0.159 | 0.218 | 0.165 | 0.206 | 0.172 | 0.220 | 0.196 | 0.255 |
| | 192 | **0.207** | **0.246** | 0.207 | 0.250 | 0.221 | 0.254 | 0.223 | 0.257 | 0.224 | 0.259 | 0.208 | 0.250 | 0.212 | 0.254 | 0.211 | 0.266 | 0.212 | 0.250 | 0.219 | 0.261 | 0.237 | 0.296 |
| | 336 | 0.265 | **0.289** | **0.262** | 0.291 | 0.278 | 0.296 | 0.278 | 0.298 | 0.278 | 0.298 | 0.264 | 0.294 | 0.272 | 0.299 | 0.267 | 0.310 | 0.269 | 0.291 | 0.280 | 0.306 | 0.283 | 0.335 |
| | 720 | 0.344 | **0.341** | **0.343** | 0.343 | 0.358 | 0.347 | 0.355 | 0.347 | 0.350 | 0.346 | 0.346 | 0.345 | 0.350 | 0.348 | 0.352 | 0.362 | 0.347 | 0.342 | 0.365 | 0.359 | 0.345 | 0.381 |
| Electricity | 96 | 0.146 | 0.236 | **0.141** | **0.235** | 0.148 | 0.240 | 0.165 | 0.259 | 0.174 | 0.259 | 0.149 | 0.245 | 0.165 | 0.274 | 0.173 | 0.275 | 0.177 | 0.272 | 0.168 | 0.272 | 0.197 | 0.282 |
| | 192 | 0.161 | **0.249** | **0.159** | 0.252 | 0.162 | 0.253 | 0.178 | 0.272 | 0.178 | 0.265 | 0.163 | 0.257 | 0.184 | 0.292 | 0.195 | 0.288 | 0.189 | 0.281 | 0.184 | 0.289 | 0.196 | 0.285 |
| | 336 | 0.177 | **0.266** | **0.173** | 0.268 | 0.178 | 0.269 | 0.189 | 0.282 | 0.196 | 0.282 | 0.183 | 0.277 | 0.195 | 0.302 | 0.206 | 0.300 | 0.205 | 0.296 | 0.198 | 0.300 | 0.209 | 0.301 |
| | 720 | 0.210 | **0.294** | **0.201** | 0.295 | 0.225 | 0.317 | 0.223 | 0.311 | 0.237 | 0.316 | 0.218 | 0.311 | 0.231 | 0.332 | 0.231 | 0.335 | 0.244 | 0.333 | 0.220 | 0.320 | 0.245 | 0.333 |
| Traffic | 96 | 0.409 | 0.270 | 0.426 | 0.276 | **0.395** | **0.268** | 0.444 | 0.315 | 0.477 | 0.305 | 0.441 | 0.286 | 0.603 | 0.343 | 0.570 | 0.310 | 0.485 | 0.324 | 0.593 | 0.321 | 0.650 | 0.396 |
| | 192 | 0.431 | 0.277 | 0.458 | 0.289 | **0.417** | **0.276** | 0.460 | 0.316 | 0.471 | 0.299 | 0.473 | 0.297 | 0.614 | 0.361 | 0.577 | 0.321 | 0.492 | 0.325 | 0.617 | 0.336 | 0.598 | 0.370 |
| | 336 | 0.446 | 0.283 | 0.486 | 0.297 | **0.433** | **0.283** | 0.471 | 0.317 | 0.485 | 0.305 | 0.487 | 0.309 | 0.643 | 0.377 | 0.588 | 0.324 | 0.510 | 0.332 | 0.629 | 0.336 | 0.605 | 0.373 |
| | 720 | 0.479 | 0.301 | 0.498 | 0.313 | **0.467** | **0.302** | 0.486 | 0.318 | 0.518 | 0.325 | 0.541 | 0.317 | 0.702 | 0.401 | 0.597 | 0.337 | 0.540 | 0.348 | 0.640 | 0.350 | 0.645 | 0.394 |
| Solar | 96 | **0.189** | **0.220** | 0.197 | 0.241 | 0.203 | 0.237 | 0.231 | 0.270 | 0.224 | 0.274 | 0.209 | 0.273 | 0.251 | 0.281 | 0.240 | 0.327 | 0.202 | 0.232 | 0.250 | 0.292 | 0.290 | 0.378 |
| | 192 | **0.226** | **0.245** | 0.231 | 0.264 | 0.233 | 0.261 | 0.303 | 0.318 | 0.262 | 0.299 | 0.235 | 0.288 | 0.277 | 0.301 | 0.263 | 0.338 | 0.239 | 0.256 | 0.296 | 0.318 | 0.320 | 0.398 |
| | 336 | **0.246** | **0.262** | 0.241 | 0.268 | 0.248 | 0.273 | 0.340 | 0.334 | 0.266 | 0.299 | 0.248 | 0.307 | 0.318 | 0.325 | 0.286 | 0.346 | 0.260 | 0.271 | 0.319 | 0.330 | 0.353 | 0.415 |
| | 720 | 0.251 | **0.265** | 0.250 | 0.281 | **0.249** | 0.275 | 0.349 | 0.337 | 0.269 | 0.297 | 0.250 | 0.299 | 0.311 | 0.328 | 0.286 | 0.347 | 0.264 | 0.273 | 0.338 | 0.337 | 0.356 | 0.413 |
| Exchange | 96 | **0.082** | **0.201** | 0.084 | 0.204 | 0.086 | 0.206 | 0.085 | 0.205 | 0.085 | 0.203 | 0.090 | 0.213 | 0.102 | 0.230 | 0.084 | 0.203 | 0.089 | 0.210 | 0.107 | 0.234 | 0.088 | 0.218 |
| | 192 | 0.174 | 0.296 | 0.177 | 0.299 | 0.177 | 0.299 | 0.177 | 0.299 | 0.177 | 0.297 | 0.186 | 0.310 | 0.195 | 0.317 | **0.171** | **0.294** | 0.188 | 0.310 | 0.226 | 0.344 | 0.176 | 0.315 |
| | 336 | 0.329 | 0.414 | 0.332 | 0.417 | 0.331 | 0.417 | 0.336 | 0.421 | 0.322 | 0.410 | 0.343 | 0.426 | 0.359 | 0.436 | 0.319 | **0.407** | 0.354 | 0.432 | 0.367 | 0.448 | **0.313** | 0.427 |
| | 720 | 0.832 | 0.686 | 0.897 | 0.711 | 0.847 | 0.691 | 0.854 | 0.670 | 0.853 | 0.694 | 0.892 | 0.713 | 0.940 | 0.738 | **0.805** | **0.677** | 0.925 | 0.730 | 0.964 | 0.746 | 0.839 | 0.695 |
| PEMS08 | 12 | **0.079** | **0.179** | 0.080 | 0.185 | **0.079** | 0.182 | 0.087 | 0.193 | 0.086 | 0.189 | 0.088 | 0.195 | 0.108 | 0.217 | 0.095 | 0.203 | 0.093 | 0.199 | 0.112 | 0.212 | 0.154 | 0.276 |
| | 24 | **0.112** | **0.215** | 0.116 | 0.224 | 0.115 | 0.219 | 0.129 | 0.236 | 0.129 | 0.231 | 0.119 | 0.225 | 0.140 | 0.244 | 0.140 | 0.254 | 0.144 | 0.248 | 0.141 | 0.238 | 0.248 | 0.353 |
| | 48 | **0.182** | 0.277 | 0.216 | 0.306 | 0.186 | **0.235** | 0.237 | 0.322 | 0.220 | 0.297 | 0.191 | 0.286 | 0.213 | 0.306 | 0.233 | 0.331 | 0.236 | 0.326 | 0.198 | 0.283 | 0.440 | 0.470 |
| | 96 | 0.253 | 0.302 | 0.324 | 0.375 | **0.221** | **0.267** | 0.423 | 0.430 | 0.375 | 0.379 | 0.293 | 0.341 | 0.350 | 0.386 | 0.382 | 0.422 | 0.417 | 0.433 | 0.320 | 0.351 | 0.674 | 0.565 |
| Flight | 96 | **0.143** | **0.248** | 0.146 | 0.254 | 0.158 | 0.271 | 0.152 | 0.263 | 0.151 | 0.260 | 0.148 | 0.260 | 0.183 | 0.301 | 0.157 | 0.263 | 0.179 | 0.294 | 0.241 | 0.357 | 0.155 | 0.260 |
| | 192 | **0.144** | **0.247** | 0.145 | 0.251 | 0.162 | 0.275 | 0.152 | 0.261 | 0.149 | 0.257 | 0.149 | 0.259 | 0.189 | 0.306 | 0.155 | 0.260 | 0.183 | 0.297 | 0.225 | 0.338 | 0.153 | 0.256 |
| | 336 | **0.153** | **0.255** | 0.155 | 0.261 | 0.178 | 0.292 | 0.163 | 0.272 | 0.159 | 0.267 | 0.161 | 0.272 | 0.206 | 0.320 | 0.165 | 0.268 | 0.291 | 0.305 | 0.292 | 0.397 | 0.162 | 0.264 |
| | 720 | **0.187** | **0.285** | 0.189 | 0.291 | 0.222 | 0.331 | 0.204 | 0.310 | 0.201 | 0.305 | 0.194 | 0.301 | 0.253 | 0.358 | 0.204 | 0.305 | 0.227 | 0.333 | 0.313 | 0.411 | 0.197 | 0.294 |
| **Avg** | | **0.305** | | **0.313** | | **0.315** | | **0.329** | | **0.322** | | **0.321** | | **0.346** | | **0.334** | | **0.333** | | **0.352** | | **0.381** | |
| **Improvement** | | | | **2.40%** | | **3.13%** | | **7.35%** | | **5.12%** | | **4.88%** | | **11.95%** | | **8.56%** | | **8.36%** | | **13.44%** | | **19.84%** | |
| **1st Count** | | **54** | | **9** | | **13** | | **1** | | **9** | | **0** | | **1** | | **5** | | **0** | | **0** | | **1** | |

the setting of the learning rate affects the model's performance. Both excessively large and excessively small learning rates lead to significant performance degradation. As the learning rate decreases, the model performance follows a typical U-shaped trend. Further analysis reveals that when the learning rate is set to 0.001, the DFGCN model achieves the best prediction performance across all datasets.

### *4.6. Ablation experiments for similarity analysis methods*

To evaluate the influence of different similarity metrics in adjacency matrix construction, we design an ablation study to compare the impact of various commonly used inter-variable similarity measures on model performance. In addition to the originally adopted Pearson Correlation, we introduce four alternative approaches, which are: (1) Cosine Similarity, which measures the angle between two variable sequences to capture directional similarity and ignores differences in magnitude; (2) Euclidean Distance, which computes spatial distance based on variable values and uses the RBF kernel to convert spatial distance into similarity, reflecting numerical closeness between variables; (3) Mahalanobis Distance, a statistical distance that accounts for the covariance structure among variables (which is a more robust characterization of overall inter-variable correlation); (4) Spearman Correlation, which evaluates monotonic relationships between variables, focusing more on

**Table 7**
Performance comparison of different similarity measures used to construct the adjacency matrix, including Pearson correlation, cosine similarity, Euclidean distance, Mahalanobis distance, and Spearman correlation. The results are reported in terms of MSE and MAE across four datasets (Exchange, Weather, Electricity, and Solar) and four prediction horizons (96, 192, 336, 720). "Sum" represents the total error sum across all horizons for each method.

| Metric | | Exchange | | Weather | | Electricity | | Solar | |
|---|---|---|---|---|---|---|---|---|---|
| | | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE |
| Pearson | 96 | 0.082 | 0.201 | 0.156 | 0.198 | 0.146 | 0.236 | 0.189 | 0.220 |
| | 192 | 0.174 | 0.296 | 0.207 | 0.246 | 0.161 | 0.249 | 0.226 | 0.245 |
| | 336 | 0.329 | 0.414 | 0.265 | 0.289 | 0.177 | 0.266 | 0.246 | 0.262 |
| | 720 | 0.832 | 0.686 | 0.344 | 0.341 | 0.210 | 0.294 | 0.251 | 0.265 |
| | Sum | 3.014 | | 2.046 | | 1.739 | | 1.904 | |
| Cosine | 96 | 0.083 | 0.202 | 0.157 | 0.199 | 0.148 | 0.238 | 0.191 | 0.222 |
| | 192 | 0.176 | 0.298 | 0.208 | 0.245 | 0.163 | 0.252 | 0.227 | 0.246 |
| | 336 | 0.329 | 0.414 | 0.267 | 0.289 | 0.180 | 0.269 | 0.248 | 0.263 |
| | 720 | 0.832 | 0.685 | 0.343 | 0.340 | 0.213 | 0.296 | 0.252 | 0.266 |
| | Sum | 3.019 | | 2.048 | | 1.759 | | 1.915 | |
| Mahalanobis | 96 | 0.082 | 0.201 | 0.160 | 0.203 | 0.146 | 0.237 | 0.188 | 0.219 |
| | 192 | 0.176 | 0.298 | 0.206 | 0.245 | 0.162 | 0.251 | 0.223 | 0.244 |
| | 336 | 0.330 | 0.415 | 0.268 | 0.290 | 0.179 | 0.268 | 0.249 | 0.264 |
| | 720 | 0.836 | 0.688 | 0.345 | 0.341 | 0.211 | 0.295 | 0.251 | 0.265 |
| | Sum | 3.026 | | 2.058 | | 1.749 | | 1.903 | |
| Euclidean | 96 | 0.083 | 0.202 | 0.161 | 0.203 | 0.146 | 0.237 | 0.190 | 0.221 |
| | 192 | 0.175 | 0.298 | 0.210 | 0.248 | 0.161 | 0.250 | 0.229 | 0.247 |
| | 336 | 0.329 | 0.414 | 0.268 | 0.290 | 0.178 | 0.267 | 0.248 | 0.262 |
| | 720 | 0.832 | 0.686 | 0.347 | 0.343 | 0.211 | 0.294 | 0.251 | 0.265 |
| | Sum | 3.019 | | 2.070 | | 1.744 | | 1.913 | |
| 90Spearman | 96 | 0.083 | 0.202 | 0.156 | 0.198 | 0.149 | 0.239 | 0.189 | 0.220 |
| | 192 | 0.175 | 0.298 | 0.208 | 0.245 | 0.164 | 0.252 | 0.227 | 0.247 |
| | 336 | 0.330 | 0.415 | 0.265 | 0.288 | 0.182 | 0.270 | 0.249 | 0.262 |
| | 720 | 0.834 | 0.687 | 0.342 | 0.340 | 0.213 | 0.296 | 0.253 | 0.267 |
| | Sum | 3.024 | | 2.042 | | 1.765 | | 1.914 | |

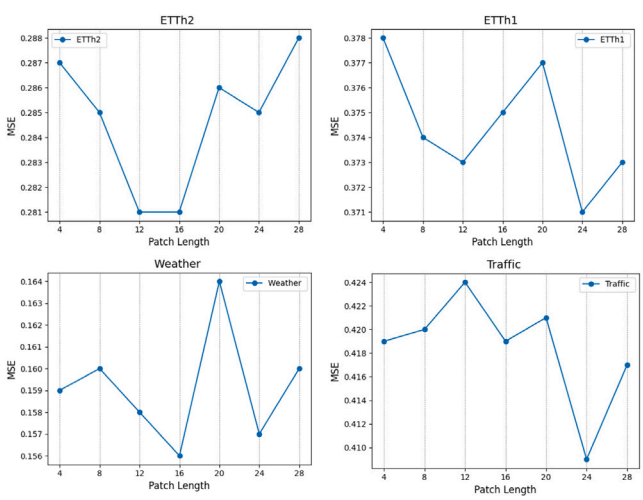


**Fig. 4.** Sensitivity analysis of patch length: The results for the ETTh1, ETTh2, Traffic, and Weather datasets with varying patch lengths, where the lookback window and prediction length are both set to 96.

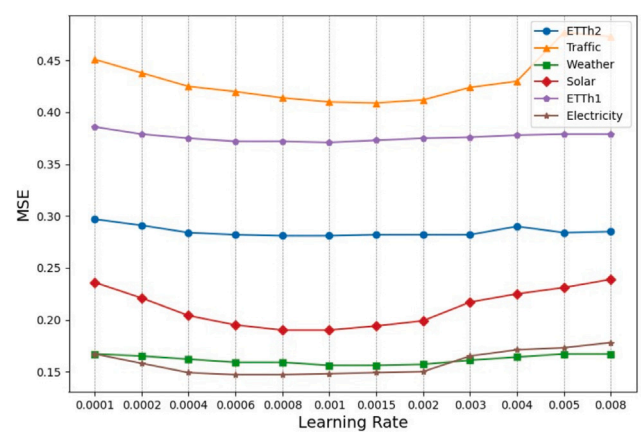


**Fig. 5.** Sensitivity analysis of the learning rate: MSE results for ETTh1, ETTh2, Weather, Traffic, Solar, and Electricity datasets in different hyperparameter learning rate settings (with input-96-predict-96 configuration).

rank order rather than absolute values. These methods characterize variable relationships from different perspectives and help to comprehensively analyze the influence of similarity construction strategies on forecasting performance.

We conduct a comprehensive comparison on four representative datasets (Exchange, Weather, Electricity, and Solar) across four forecasting horizons (96, 192, 336, and 720), using MSE and MAE as evaluation metrics. As shown in Table 7, we evaluate the forecasting performance of different similarity metrics under various experimental settings and report the cumulative errors for each experiment group. The results indicate that the choice of similarity metric has a relatively minor effect on overall model performance, with prediction results showing no significant variation. This suggests that the model demonstrates strong robustness to the selection of similarity measures. Notably, Pearson Correlation consistently achieves the best or near-best performance in most cases, exhibiting particularly high stability on the Electricity and Solar datasets. Therefore, we retain Pearson Correlation as the default similarity metric, with its effectiveness in capturing inter-series dependencies empirically validated.

### 4.7. Univariate time series forecasting

To further demonstrate the superiority of the DFGCN model, we

**Table 8**
Univariate time series forecasting results with ETT dataset. The best results are in bold. We fixed the input length to 96 for all models, with prediction lengths set to 96, 192, 336, and 720.

| Models | | DFGCN | | iTranformer | | BasisFormer | | PatchTST | | MSGNet | | CrossGNN | | FourierGNN | | TimesNet | | Dlinear | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Metric | | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE |
| ETTh2 | 96 | **0.125** | **0.270** | 0.134 | 0.286 | 0.139 | 0.290 | 0.127 | 0.273 | 0.144 | 0.295 | 0.126 | 0.271 | 0.136 | 0.285 | 0.140 | 0.292 | 0.128 | 0.271 |
| | 192 | **0.170** | **0.322** | 0.181 | 0.336 | 0.179 | 0.332 | 0.178 | 0.329 | 0.190 | 0.345 | 0.174 | 0.327 | 0.184 | 0.337 | 0.191 | 0.345 | 0.181 | 0.327 |
| | 336 | **0.207** | **0.362** | 0.217 | 0.372 | 0.212 | 0.367 | 0.220 | 0.373 | 0.221 | 0.377 | 0.217 | 0.371 | 0.221 | 0.377 | 0.238 | 0.392 | 0.230 | 0.377 |
| | 720 | **0.225** | **0.381** | 0.240 | 0.391 | **0.225** | **0.381** | 0.250 | 0.403 | 0.228 | 0.384 | 0.232 | 0.388 | 0.240 | 0.393 | 0.251 | 0.399 | 0.318 | 0.460 |
| ETTh1 | 96 | **0.054** | **0.178** | 0.061 | 0.189 | 0.055 | 0.179 | 0.056 | 0.179 | 0.056 | 0.180 | 0.058 | 0.188 | 0.058 | 0.183 | 0.057 | 0.185 | 0.060 | 0.181 |
| | 192 | 0.071 | 0.205 | 0.072 | 0.205 | 0.073 | 0.204 | 0.075 | 0.209 | **0.070** | **0.203** | 0.075 | 0.215 | 0.074 | 0.207 | 0.076 | 0.213 | 0.083 | 0.214 |
| | 336 | **0.083** | **0.226** | 0.088 | 0.231 | 0.086 | 0.227 | 0.092 | 0.238 | 0.086 | 0.227 | 0.090 | 0.238 | 0.087 | 0.229 | 0.087 | 0.231 | 0.103 | 0.249 |
| | 720 | **0.081** | **0.226** | 0.082 | **0.226** | 0.095 | 0.237 | 0.097 | 0.245 | 0.091 | 0.239 | 0.086 | 0.233 | 0.091 | 0.237 | 0.084 | 0.230 | 0.150 | 0.316 |
| ETTm1 | 96 | **0.028** | **0.125** | 0.029 | 0.127 | 0.030 | 0.128 | 0.029 | 0.126 | **0.028** | **0.125** | 0.030 | 0.130 | 0.030 | 0.129 | 0.029 | 0.127 | 0.033 | 0.133 |
| | 192 | **0.043** | **0.158** | 0.049 | 0.168 | 0.044 | 0.160 | **0.043** | **0.158** | 0.044 | 0.159 | 0.044 | 0.159 | 0.045 | 0.161 | 0.046 | 0.162 | 0.059 | 0.181 |
| | 336 | **0.057** | 0.184 | 0.061 | 0.190 | 0.059 | 0.186 | **0.057** | **0.183** | 0.059 | 0.187 | 0.060 | 0.189 | 0.059 | 0.187 | 0.058 | 0.186 | 0.074 | 0.202 |
| | 720 | **0.079** | **0.217** | 0.082 | 0.219 | 0.083 | 0.221 | 0.081 | 0.218 | 0.083 | 0.220 | 0.082 | 0.224 | 0.083 | 0.220 | 0.081 | 0.219 | 0.102 | 0.239 |
| ETTm2 | 96 | 0.065 | **0.180** | 0.068 | 0.188 | 0.067 | 0.187 | **0.063** | **0.180** | 0.073 | 0.203 | 0.070 | 0.193 | 0.065 | 0.183 | 0.075 | 0.201 | 0.070 | 0.191 |
| | 192 | **0.098** | **0.230** | 0.107 | 0.244 | 0.101 | 0.236 | **0.098** | 0.233 | 0.107 | 0.249 | 0.109 | 0.248 | 0.100 | 0.234 | 0.106 | 0.247 | 0.103 | 0.238 |
| | 336 | **0.127** | **0.269** | 0.144 | 0.288 | 0.129 | 0.273 | 0.132 | 0.275 | 0.142 | 0.289 | 0.136 | 0.282 | 0.130 | 0.273 | 0.141 | 0.289 | 0.134 | 0.277 |
| | 720 | **0.181** | **0.328** | 0.185 | 0.337 | 0.181 | 0.331 | 0.186 | 0.335 | 0.198 | 0.347 | 0.187 | 0.335 | 0.182 | 0.330 | 0.189 | 0.340 | 0.187 | 0.331 |
| **Avg** | | **0.174** | | **0.181** | | **0.178** | | **0.179** | | **0.183** | | **0.180** | | **0.180** | | **0.185** | | **0.194** | |
| **Improvement** | | | | **4.17%** | | **2.49%** | | **3.24%** | | **5.03%** | | **3.67%** | | **3.39%** | | **5.96%** | | **10.43%** | |

**Table 9**
Compares the results of MS and S on the Traffic and Electricity datasets. Here, S represents a univariate input predicting a univariate output, while MS represents a multivariate input predicting a univariate output. The input length is fixed at 96, with prediction lengths set to 96, 192, 336, and 720.

| Models | | DFGCN | | | | iTransformer | | | | PatchTST | | | | Basisformer | | | | GrossGNN | | | | FourierGNN | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | S | | MS | | S | | MS | | S | | MS | | S | | MS | | S | | MS | | S | | MS | |
| Metric | | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE |
| Electricity | 96 | 0.295 | 0.383 | 0.253 | 0.265 | 0.259 | 0.366 | 0.279 | 0.381 | 0.303 | 0.390 | 0.296 | 0.389 | 0.328 | 0.406 | 0.303 | 0.396 | 0.344 | 0.422 | 0.330 | 0.420 | 0.339 | 0.413 | 0.314 | 0.407 |
| | 192 | 0.327 | 0.398 | 0.282 | 0.377 | 0.304 | 0.391 | 0.315 | 0.401 | 0.321 | 0.398 | 0.318 | 0.400 | 0.344 | 0.414 | 0.333 | 0.409 | 0.334 | 0.411 | 0.328 | 0.413 | 0.369 | 0.426 | 0.346 | 0.421 |
| | 336 | 0.369 | 0.424 | 0.320 | 0.403 | 0.357 | 0.425 | 0.363 | 0.432 | 0.373 | 0.434 | 0.380 | 0.444 | 0.413 | 0.452 | 0.368 | 0.430 | 0.390 | 0.448 | 0.397 | 0.455 | 0.422 | 0.464 | 0.391 | 0.450 |
| | 720 | 0.430 | 0.476 | 0.336 | 0.426 | 0.404 | 0.462 | 0.437 | 0.488 | 0.434 | 0.481 | 0.444 | 0.497 | 0.501 | 0.513 | 0.451 | 0.493 | 0.455 | 0.497 | 0.425 | 0.483 | 0.543 | 0.532 | 0.474 | 0.507 |
| Traffic | 96 | 0.149 | 0.218 | 0.139 | 0.204 | 0.158 | 0.247 | 0.189 | 0.282 | 0.183 | 0.272 | 0.188 | 0.279 | 0.182 | 0.277 | 0.182 | 0.276 | 0.248 | 0.335 | 0.243 | 0.328 | 0.187 | 0.269 | 0.176 | 0.263 |
| | 192 | 0.146 | 0.215 | 0.136 | 0.203 | 0.147 | 0.230 | 0.178 | 0.271 | 0.176 | 0.262 | 0.185 | 0.269 | 0.177 | 0.267 | 0.167 | 0.260 | 0.222 | 0.307 | 0.219 | 0.301 | 0.181 | 0.265 | 0.176 | 0.264 |
| | 336 | 0.145 | 0.218 | 0.137 | 0.210 | 0.146 | 0.234 | 0.177 | 0.275 | 0.173 | 0.261 | 0.179 | 0.267 | 0.175 | 0.273 | 0.162 | 0.256 | 0.213 | 0.301 | 0.205 | 0.293 | 0.176 | 0.265 | 0.170 | 0.260 |
| | 720 | 0.163 | 0.229 | 0.151 | 0.227 | 0.172 | 0.259 | 0.190 | 0.284 | 0.200 | 0.288 | 0.241 | 0.307 | 0.186 | 0.272 | 0.174 | 0.263 | 0.232 | 0.319 | 0.227 | 0.312 | 0.199 | 0.286 | 0.191 | 0.279 |
| **Avg** | | **0.287** | | **0.254** (↑12.99%) | | **0.285** | | **0.309**(↓7.76%) | | **0.309** | | **0.318**(↓2.83%) | | **0.324** | | **0.308**(↑5.19%) | | **0.342** | | **0.336**(↑1.78%) | | **0.334** | | **0.318**(↑5.03%) | |

compare it with eight benchmark models on univariate time series forecasting tasks. Consistent with our approach in PatchTST, we utilize the ETT dataset, treating the target feature 'oil temperature' as a univariate sequence for prediction. The experimental results are presented in Table 8. From these results, it can be observed that the DFGCN model achieves performance improvements ranging from 2.49% to 10.43% compared to the other baseline models. This indicates that the DFGCN model is capable of capturing more potential features in the time dimension, thereby leading to superior forecasting performance.

### *4.8. Demonstrating the effectiveness of cross-variable interaction*

To explore the efficiency of different models in cross-variable interaction learning, we validate our approach on the electricity and traffic datasets, which contain a large number of variables. We compare the proposed DFGCN model with the top five baseline models from the main experiments, namely iTransformer, PatchTST, Basisformer, GrossGNN, and FourierGNN. Consistent with existing studies, we perform univariate prediction of the target variable (denoted as OT) for univariate scenarios (S) and multivariate prediction of the target variable for multivariate scenarios (denoted as MS). The MS task aims to improve the prediction accuracy of the target variable by leveraging multiple covariates in addition to the target variable itself. For example, in the electricity dataset, the target variable 'electricity load' is influenced by covariates such as weather conditions and time of use.

The experimental results are presented in Table 9. These results indicate that the DFGCN model can significantly leverage covariates to enhance the prediction performance of the target variable. This suggests that the DFGCN model is more efficient at capturing latent relevant information from covariates. Furthermore, it is noteworthy that the performance of the iTransformer model exhibits a significant drop after the introduction of covariates. This is attributed to the model's inability to effectively filter out irrelevant information, which results in the introduction of substantial noise and subsequently negatively impacts the prediction performance.

In addition, we observe an unusual phenomenon in our experiments on the Traffic dataset: some models perform better when the forecasting horizon is 336 compared to when it is 96 and 192. We believe this trend is not accidental, but is closely related to the dataset's inherent periodic characteristics, the model training trajectory, and the error evaluation mechanism. Similar observations have also been reported in previous studies. For instance, the original papers of PatchTST [26], TimeXer [61], and TTM [62] all indicate that longer forecasting horizons on the Traffic dataset may result in lower prediction errors. We speculate that this phenomenon may result from the fact that longer time windows better capture the stable periodic structures present in traffic data, thereby enhancing the model's ability to learn long-term trends. Additionally, the differences in optimization, which are due to independently training models under different forecasting horizons, may lead to better performance at certain longer horizons. Furthermore, evaluation metrics such as MSE and MAE may become smoother under longer sequences. This is because they are more influenced by overarching trends, which could further amplify this phenomenon. Therefore, we believe this trend reflects the unique characteristics of the Traffic dataset rather than any abnormality in the model's generalization ability.

**Table 10**
Computational efficiency analysis on the Electricity and Traffic datasets. The symbols "M", "M", "S" and "M" are used to denote the units for "Memory", "Para", "Time" and "FLOPs", respectively, representing million bytes, million bytes, seconds and million floating-point operations, respectively. "–" means the calculation failed. Smaller values indicate better performance.

| | Models | Electricity | | | | | | Traffic | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | Memory(M) | Para(M) | Time (s) | FLOPs(M) | MSE | MAE | Memory(M) | Para(M) | Time (s) | FLOPs(M) | MSE | MAE |
| MLP | Dlinear | 502 | 0.07 | 5 | 21.0 | 0.197 | 0.282 | 594 | 0.07 | 7 | 56.4 | 0.650 | 0.393 |
| CNN | TimesNet | 4562 | 150.31 | 297 | 4 621 783.8 | 0.168 | 0.272 | 7806 | 301.69 | 386 | 9 302 680.2 | 0.593 | 0.321 |
| Transformer | PatchTST | 5190 | 0.55 | 137 | 1630.3 | 0.172 | 0.259 | 10 578 | 0.55 | 510 | 4378.0 | 0.477 | 0.305 |
| | iTransformer | 1586 | 4.83 | 15 | 1569.8 | 0.148 | 0.240 | 5952 | 6.41 | 49 | 5548.7 | 0.395 | 0.268 |
| | BasisFormer | 958 | 0.66 | 9 | – | 0.165 | 0.259 | 1720 | 0.66 | 12 | – | 0.444 | 0.315 |
| | Leddam | 4732 | 3.24 | 15 | 1583.3 | 0.141 | 0.235 | 3740 | 3.24 | 31 | 4969.5 | 0.426 | 0.276 |
| GNN | FourierGNN | 5896 | 0.19 | 86 | 71.2 | 0.177 | 0.272 | 17 656 | 0.19 | 189 | 191.1 | 0.485 | 0.324 |
| | CrossGNN | 5392 | 0.02 | 39 | 213.3 | 0.173 | 0.275 | 7058 | 0.02 | 74 | 575.4 | 0.570 | 0.310 |
| | MSGNet | 8948 | 18.90 | 349 | 336 559.5 | 0.165 | 0.274 | 9984 | 27.65 | 507 | 392 701.6 | 0.603 | 0.343 |
| | DyGraphformer | 5108 | 1.15 | 70 | 2610.7 | 0.149 | 0.245 | 13 588 | 5.01 | 289 | 33 546.2 | 0.441 | 0.286 |
| | **DFGCN (Ours)** | 1134 | 3.72 | 16 | 1297.5 | 0.146 | 0.236 | 2554 | 3.73 | 27 | 3726.8 | 0.409 | 0.270 |

### *4.9. Computational efficiency analysis*

In this section, we compare the training overhead of the proposed DFGCN model with that of other baseline models across two representative datasets. The models are comprehensively evaluated using four metrics: (1) the number of model parameters (Para), which indicates the learnable capacity; (2) GPU memory usage (Memory), reflecting the GPU resource cost during training or inference; (3) floating-point operations (FLOPs), representing the computational complexity; and (4) training time per epoch.

As shown in Table 10, although DFGCN has a relatively larger number of parameters, it demonstrates significant advantages in key metrics such as memory usage, training time, and FLOPs. Additionally, although iTransformer and Leddam also have a relatively large number of parameters, they outperform most lightweight models in terms of training time. This indicates that the size of the model's parameters does not directly determine the model's efficiency. Specifically, compared with PatchTST and DyGraphformer, although DFGCN has more parameters, it achieves better performance in terms of memory consumption, training time, and FLOPs, demonstrating higher computational efficiency. Meanwhile, compared to the high-performing Leddam model, DFGCN significantly reduces memory consumption while maintaining a comparable training time, demonstrating superior resource efficiency. It is worth noting that among all graph neural network-based models, DFGCN achieves the shortest training time, significantly outperforming other graph-based approaches such as FourierGNN, CrossGNN, and MSGNet. This demonstrates that DFGCN maintains strong graph modeling capabilities while also ensuring high efficiency. We believe that this improvement in efficiency mainly benefits from the following design optimizations: (1) Top-K sparse graph convolution strategy: By dynamically constructing sparse adjacency structures, it effectively avoids the computational and memory burden caused by redundant connections in dense graphs, significantly reducing FLOPs and memory usage. (2) Dual-flow decoupled parallel architecture: By separately modeling the temporal and variable dimensions and enabling parallel training paths, it substantially shortens both the forward and backward propagation processes compared to traditional single-flow deep stacking structures, thereby improving overall training efficiency.

In summary, DFGCN achieves a well-balanced trade-off between model accuracy and computational efficiency. Even with a slightly larger number of parameters, it still delivers outstanding performance in terms of lower memory usage, faster training, and reduced computational cost, highlighting its potential and advantages for real-world deployment.

### *4.10. Effects of different look-back windows*

To explore the gain in prediction performance associated with input length, we conduct a sensitivity analysis on the window size across six benchmark datasets: Electricity, Traffic, Flight, Solar Energy, ETTh1, and Weather. The experimental results are reported in Table 11. As observed, with the increase in input length, the MSE and MAE evaluation metrics of the DFGCN model show a gradual decline across all datasets, indicating an improvement in prediction performance. Specifically, when the input length increases from 96 to 192, the model's performance improves by an average of 3.55%; when the input length further increases from 192 to 336, the performance improves by an additional 2.18%. This result suggests that the proposed DFGCN model benefits from longer input lengths, implying that the method can learn more latent features from extended time series data.

### *4.11. Visualization of forecasting results of different models*

To provide a more intuitive evaluation of the proposed DFGCN model's prediction performance in comparison to baseline models, we conduct a visual analysis of the prediction results on the Electricity dataset. Fig. 6 showcases a comparison between the predicted results and the actual values for all baseline models, alongside those of the DFGCN model. As observed, the predictions from the DFGCN model are closer to the actual values, thereby demonstrating superior prediction ability relative to the baseline models.

## 5. Conclusion

In this paper, we address the challenge of modeling intra-series and inter-series correlations in MTSF tasks by proposing a decoupled DFGCN, which effectively resolves the issue of feature interference in existing single-flow architectures. By introducing a dual-flow structure, DFGCN separately models the temporal and variable dimensions to capture the heterogeneous dependencies in multivariate time series data. Specifically, the dual-flow embedding layer represents the features of time and variables, laying the foundation for modeling both intra-series and inter-series dependencies. The DSGCN sparsifies the graph based on Pearson correlation weights, effectively learning local dependencies and reducing computational overhead. The dual-flow global attention layer further captures global information, complementing the local features. Experimental results show that DFGCN exhibits outstanding prediction performance across several real-world datasets. Additionally, the model's robustness and the effectiveness of its components, such as the contributions of the time flow, variable flow, and GCN components, are thoroughly validated in the ablation studies, further confirming the rationality of the dual-flow structure

**Table 11**
MTSF results with varying look-back window in DFGCN. Look back window size: $L \in \{48, 96, 192, 336\}$.

| Input length L | | 336 | | 192 | | 96 | | 48 | |
|---|---|---|---|---|---|---|---|---|---|
| Metric | | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE |
| Weather | 96 | 0.145 | 0.191 | 0.156 | 0.200 | 0.156 | 0.198 | 0.197 | 0.235 |
| | 192 | 0.189 | 0.231 | 0.196 | 0.237 | 0.207 | 0.246 | 0.238 | 0.268 |
| | 336 | 0.240 | 0.273 | 0.252 | 0.279 | 0.265 | 0.289 | 0.294 | 0.308 |
| | 720 | 0.314 | 0.326 | 0.327 | 0.332 | 0.344 | 0.341 | 0.371 | 0.358 |
| Electricity | 96 | 0.130 | 0.223 | 0.133 | 0.225 | 0.146 | 0.236 | 0.181 | 0.263 |
| | 192 | 0.151 | 0.242 | 0.152 | 0.242 | 0.161 | 0.249 | 0.189 | 0.270 |
| | 336 | 0.166 | 0.258 | 0.168 | 0.258 | 0.177 | 0.266 | 0.207 | 0.287 |
| | 720 | 0.194 | 0.281 | 0.197 | 0.283 | 0.210 | 0.294 | 0.239 | 0.311 |
| Traffic | 96 | 0.368 | 0.250 | 0.378 | 0.254 | 0.409 | 0.270 | 0.508 | 0.309 |
| | 192 | 0.390 | 0.259 | 0.401 | 0.264 | 0.431 | 0.277 | 0.509 | 0.308 |
| | 336 | 0.403 | 0.266 | 0.416 | 0.272 | 0.446 | 0.283 | 0.528 | 0.315 |
| | 720 | 0.432 | 0.283 | 0.445 | 0.289 | 0.479 | 0.301 | 0.565 | 0.335 |
| ETTh1 | 96 | 0.364 | 0.391 | 0.370 | 0.391 | 0.371 | 0.389 | 0.386 | 0.391 |
| | 192 | 0.406 | 0.418 | 0.408 | 0.413 | 0.421 | 0.416 | 0.437 | 0.424 |
| | 336 | 0.429 | 0.431 | 0.444 | 0.428 | 0.462 | 0.438 | 0.489 | 0.450 |
| | 720 | 0.451 | 0.468 | 0.467 | 0.471 | 0.460 | 0.454 | 0.487 | 0.467 |
| Solar | 96 | 0.179 | 0.218 | 0.173 | 0.212 | 0.189 | 0.220 | 0.402 | 0.377 |
| | 192 | 0.191 | 0.231 | 0.204 | 0.239 | 0.226 | 0.245 | 0.365 | 0.354 |
| | 336 | 0.201 | 0.240 | 0.220 | 0.252 | 0.246 | 0.262 | 0.380 | 0.359 |
| | 720 | 0.204 | 0.243 | 0.223 | 0.250 | 0.251 | 0.265 | 0.376 | 0.355 |
| Flight | 96 | 0.121 | 0.222 | 0.126 | 0.228 | 0.143 | 0.248 | 0.163 | 0.270 |
| | 192 | 0.127 | 0.228 | 0.132 | 0.233 | 0.144 | 0.247 | 0.160 | 0.264 |
| | 336 | 0.139 | 0.239 | 0.144 | 0.244 | 0.153 | 0.255 | 0.169 | 0.272 |
| | 720 | 0.181 | 0.279 | 0.186 | 0.281 | 0.187 | 0.285 | 0.202 | 0.300 |
| **Avg** | | **0.275**(↑**2.18%**) | | **0.281**(↑**3.55%**) | | **0.291**(↑**18.9%**) | | **0.346** | |

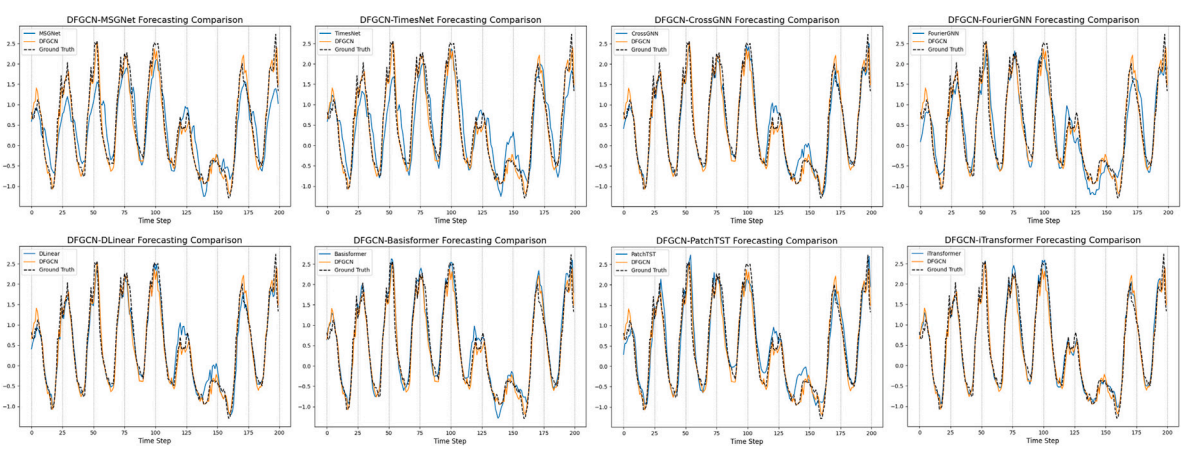


**Fig. 6.** Visualization of input-96-predict-96 results on the Electricity dataset.

design. Computational efficiency analysis demonstrates that the DFGCN model strikes a good balance between efficiency and performance. Overall, the DFGCN model proposed in this paper effectively addresses the complex dependency modeling issues in MTSF, and its superiority and robustness have been validated through experiments on multiple datasets. This research provides new ideas and methods for MTSF tasks.

Although DFGCN demonstrates strong forecasting capabilities across multiple benchmarks, it still has several limitations. First, due to the use of patch division, it primarily models temporal features at a relatively coarse granularity. While this strategy improves computational efficiency, it limits the model's ability to capture fine-grained temporal variations within individual patches. This may lead to performance bottlenecks when dealing with data containing complex local dynamics. Second, the model is sensitive to certain hyperparameters (e.g., patch length), requiring task-specific tuning across different datasets to achieve optimal performance. In addition, although the dual-stream structure enhances the modeling of both variable and temporal dependencies, it introduces additional parameters, thereby increasing the overall training cost to some extent. Future work will focus on improving fine-grained modeling capabilities, enhancing hyperparameter robustness, and further optimizing computational efficiency.

## CRediT authorship contribution statement

**Junjie Ye:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Jinhong Li:** Writing – review & editing, Visualization, Software, Data curation. **Rui Su:** Writing – original draft, Data curation. **Sen**

**Yang:** Software, Formal analysis, Data curation. **Yaqun Huang:** Supervision, Funding acquisition. **Chunna Zhao:** Writing – review & editing, Supervision, Project administration, Funding acquisition.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

This research is supported by the National Natural Science Foundation of China (61862062, 61104035), Yunnan Fundamental Research Projects, China (202401AT070471), Scientific Research and Innovation Project of Postgraduate Students in the Academic Degree of YunNan University, China (KC-242410317).

## Data availability

Data will be made available on request.

## References

[1] B. Li, W. Cui, L. Zhang, C. Zhu, W. Wang, I. Tsang, J.T. Zhou, DifFormer: Multi-resolutional differencing transformer with dynamic ranging for time series analysis, IEEE Trans. Pattern Anal. Mach. Intell. (2023).
[2] R.P. Masini, M.C. Medeiros, E.F. Mendes, Machine learning advances for time series forecasting, J. Econ. Surv. 37 (1) (2023) 76–111.
[3] C. Wang, Z. Li, J. Yu, L. Xu, High-low frequency dynamic interactive fusion network for multivariate time series forecasting, Knowl.-Based Syst. (2025) 112951.
[4] N. Banerjee, A. Morton, K. Akartunalı, Passenger demand forecasting in scheduled transportation, European J. Oper. Res. 286 (3) (2020) 797–810.
[5] B. Kumar, N. Yadav, et al., A novel hybrid model combining $\beta$SARMA and LSTM for time series forecasting, Appl. Soft Comput. 134 (2023) 110019.
[6] V.K.R. Chimmula, L. Zhang, Time series forecasting of COVID-19 transmission in Canada using LSTM networks, Chaos Solitons Fractals 135 (2020) 109864.
[7] R.P. Masini, M.C. Medeiros, E.F. Mendes, Machine learning advances for time series forecasting, J. Econ. Surv. 37 (1) (2023) 76–111.
[8] L. Han, H.-J. Ye, D.-C. Zhan, The capacity and robustness trade-off: Revisiting the channel independent strategy for multivariate time series forecasting, IEEE Trans. Knowl. Data Eng. (2024).
[9] J. Ye, W. Li, Z. Zhang, T. Zhu, L. Sun, B. Du, MvTS-library: An open library for deep multivariate time series forecasting, Knowl.-Based Syst. 283 (2024) 111170.
[10] G. Li, J.J. Jung, Deep learning for anomaly detection in multivariate time series: Approaches, applications, and challenges, Inf. Fusion 91 (2023) 93–102.
[11] Y. Wang, Y. Xu, J. Yang, M. Wu, X. Li, L. Xie, Z. Chen, Fully-connected spatial-temporal graph for multivariate time-series data, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 38, 2024, pp. 15715–15724.
[12] W. Cai, Y. Liang, X. Liu, J. Feng, Y. Wu, Msgnet: Learning multi-scale inter-series correlations for multivariate time series forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 38, 2024, pp. 11141–11149.
[13] K. Rasul, C. Seward, I. Schuster, R. Vollgraf, Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting, in: International Conference on Machine Learning, PMLR, 2021, pp. 8857–8868.
[14] C.N. Babu, B.E. Reddy, A moving-average filter based hybrid ARIMA–ANN model for forecasting time series data, Appl. Soft Comput. 23 (2014) 27–38.
[15] D. Cao, Y. Wang, J. Duan, C. Zhang, X. Zhu, C. Huang, Y. Tong, B. Xu, J. Bai, J. Tong, et al., Spectral temporal graph neural network for multivariate time-series forecasting, Adv. Neural Inf. Process. Syst. 33 (2020) 17766–17778.
[16] W. Weng, J. Fan, H. Wu, Y. Hu, H. Tian, F. Zhu, J. Wu, A decomposition dynamic graph convolutional recurrent network for traffic forecasting, Pattern Recognit. 142 (2023) 109670.
[17] L. Shen, Y. Wei, Y. Wang, H. Qiu, FDNet: Focal Decomposed Network for efficient, robust and practical time series forecasting, Knowl.-Based Syst. 275 (2023) 110666.
[18] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long, Timesnet: Temporal 2d-variation modeling for general time series analysis, in: The Eleventh International Conference on Learning Representations, 2023.
[19] Y. Liu, C. Li, J. Wang, M. Long, Koopa: Learning non-stationary time series dynamics with koopman predictors, Adv. Neural Inf. Process. Syst. 36 (2023) 12271–12290.
[20] K. Yi, Q. Zhang, W. Fan, S. Wang, P. Wang, H. He, N. An, D. Lian, L. Cao, Z. Niu, Frequency-domain MLPs are more effective learners in time series forecasting, Adv. Neural Inf. Process. Syst. 36 (2024).
[21] Z. Ni, H. Yu, S. Liu, J. Li, W. Lin, Basisformer: Attention-based time series forecasting with learnable and interpretable basis, Adv. Neural Inf. Process. Syst. 36 (2024).
[22] Y. Chen, S. Liu, J. Yang, H. Jing, W. Zhao, G. Yang, A Joint Time-Frequency Domain Transformer for multivariate time series forecasting, Neural Netw. 176 (2024) 106334.
[23] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, M. Long, iTransformer: Inverted transformers are effective for time series forecasting, in: The Twelfth International Conference on Learning Representations, 2024, URL: https://openreview.net/forum?id=JePfAI8fah.
[24] X. Zhou, W. Wang, W. Buntine, S. Qu, A. Sriramulu, W. Tan, C. Bergmeir, Scalable transformer for high dimensional multivariate time series forecasting, in: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, 2024, pp. 3515–3526.
[25] W. Han, T. Zhu, L. Chen, H. Ning, Y. Luo, Y. Wan, Mcformer: multivariate time series forecasting with mixed-channels transformer, IEEE Internet Things J. 11 (17) (2024) 28320–28329.
[26] Y. Nie, N.H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: Long-term forecasting with transformers, in: The Eleventh International Conference on Learning Representations, 2023, URL: https://openreview.net/forum?id=Jbdc0vTOcol.
[27] A. Zeng, M. Chen, L. Zhang, Q. Xu, Are transformers effective for time series forecasting? in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 11121–11128.
[28] G. Yu, J. Zou, X. Hu, A.I. Aviles-Rivero, J. Qin, S. Wang, Revitalizing multivariate time series forecasting: Learnable decomposition with inter-series dependencies and intra-series variations modeling, in: International Conference on Machine Learning, PMLR, 2024, pp. 57818–57841.
[29] S. Han, Y. Xun, J. Cai, H. Yang, Y. Li, DyGraphformer: Transformer combining dynamic spatio-temporal graph network for multivariate time series forecasting, Neural Netw. 181 (2025) 106776.
[30] J. Liu, C. Liu, G. Woo, Y. Wang, B. Hooi, C. Xiong, D. Sahoo, UniTST: Effectively modeling inter-series and intra-series dependencies for multivariate time series forecasting, in: NeurIPS Workshop on Time Series in the Age of Large Models, 2024, URL: https://openreview.net/forum?id=QJE0haPcY5.
[31] K. Yi, Q. Zhang, H. He, K. Shi, L. Hu, N. An, Z. Niu, Deep coupling network for multivariate time series forecasting, ACM Trans. Inf. Syst. 42 (5) (2024) 1–28.
[32] M. Jin, H.Y. Koh, Q. Wen, D. Zambon, C. Alippi, G.I. Webb, I. King, S. Pan, A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection, IEEE Trans. Pattern Anal. Mach. Intell. (2024).
[33] H. Zhao, Y. Wang, J. Duan, C. Huang, D. Cao, Y. Tong, B. Xu, J. Bai, J. Tong, Q. Zhang, Multivariate time-series anomaly detection via graph attention network, in: 2020 IEEE International Conference on Data Mining, ICDM, IEEE, 2020, pp. 841–850.
[34] Z. Dong, R. Jiang, H. Gao, H. Liu, J. Deng, Q. Wen, X. Song, Heterogeneity-informed meta-parameter learning for spatiotemporal time series forecasting, in: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 631–641.
[35] Y. Zhang, J. Yan, Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting, in: The Eleventh International Conference on Learning Representations, 2023.
[36] K. Yi, Q. Zhang, W. Fan, H. He, L. Hu, P. Wang, N. An, L. Cao, Z. Niu, FourierGNN: Rethinking multivariate time series forecasting from a pure graph perspective, Adv. Neural Inf. Process. Syst. 36 (2024).
[37] A. Vaswani, Attention is all you need, Adv. Neural Inf. Process. Syst. (2017).
[38] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: International Conference on Learning Representations, 2017, URL: https://openreview.net/forum?id=SJU4ayYgl.
[39] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin, Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting, in: International Conference on Machine Learning, PMLR, 2022, pp. 27268–27286.
[40] W. Chen, W. Wang, B. Peng, Q. Wen, T. Zhou, L. Sun, Learning to rotate: Quaternion transformer for complicated periodical time series forecasting, in: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2022, pp. 146–156.
[41] Y. Zhao, Z. Ma, T. Zhou, M. Ye, L. Sun, Y. Qian, GCformer: an efficient solution for accurate and scalable long-term multivariate time series forecasting, in: Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, 2023, pp. 3464–3473.
[42] R.-G. Cirstea, C. Guo, B. Yang, T. Kieu, X. Dong, S. Pan, Triformer: Triangular, variable-specific attentions for long sequence multivariate time series forecasting, in: The Thirty-First International Joint Conference on Artificial Intelligence, 2022, pp. 1994–2001.
[43] B. Li, W. Cui, L. Zhang, C. Zhu, W. Wang, I.W. Tsang, J.T. Zhou, Difformer: Multi-resolutional differencing transformer with dynamic ranging for time series analysis, IEEE Trans. Pattern Anal. Mach. Intell. 45 (11) (2023) 13586–13598.

[44] J. Gao, W. Hu, Y. Chen, Client: Cross-variable linear integrated enhanced transformer for multivariate long-term time series forecasting, 2023, pp. arXiv–2305, arXiv e-prints.

[45] X. Wang, T. Zhou, Q. Wen, J. Gao, B. Ding, R. Jin, CARD: Channel aligned robust blend transformer for time series forecasting, in: The Twelfth International Conference on Learning Representations, 2024, URL: https://openreview.net/forum?id=MJksrOhurE.

[46] Z. Zhang, L. Meng, Y. Gu, SageFormer: Series-aware framework for long-term multivariate time series forecasting, IEEE Internet Things J. (2024) http://dx.doi.org/10.1109/JIOT.2024.3363451.

[47] Q. Huang, L. Shen, R. Zhang, S. Ding, B. Wang, Z. Zhou, Y. Wang, Crossgnn: Confronting noisy multivariate time series via cross interaction refinement, Adv. Neural Inf. Process. Syst. 36 (2023) 46885–46902.

[48] F. Scarselli, M. Gori, A.C. Tsoi, M. Hagenbuchner, G. Monfardini, The graph neural network model, IEEE Trans. Neural Netw. 20 (1) (2008) 61–80.

[49] Y. Jiang, X. Li, Y. Chen, S. Liu, W. Kong, A.F. Lentzakis, G. Cong, SAGDFN: A scalable adaptive graph diffusion forecasting network for multivariate time series forecasting, in: 2024 IEEE 40th International Conference on Data Engineering, ICDE, IEEE Computer Society, 2024, pp. 1255–1268, http://dx.doi.org/10.1109/ICDE60146.2024.00101.

[50] I. Marisca, C. Alippi, F.M. Bianchi, Graph-based forecasting with missing data through spatiotemporal downsampling, in: Forty-First International Conference on Machine Learning, 2024, URL: https://openreview.net/forum?id=uYIFQOtb58.

[51] J. Fan, W. Weng, H. Tian, H. Wu, F. Zhu, J. Wu, RGDAN: A random graph diffusion attention network for traffic prediction, Neural Netw. 172 (2024) 106093.

[52] M. Ma, J. Hu, C.S. Jensen, F. Teng, P. Han, Z. Xu, T. Li, Learning time-aware graph structures for spatially correlated time series forecasting, in: 2024 IEEE 40th International Conference on Data Engineering, ICDE, IEEE, 2024, pp. 4435–4448.

[53] M. Mao, R. Zhang, H. Zheng, T. Ma, Y. Peng, E. Ding, B. Zhang, S. Han, et al., Dual-stream network for visual recognition, Adv. Neural Inf. Process. Syst. 34 (2021) 25346–25358.

[54] Y. Mou, X. Jiang, K. Xu, T. Sun, Z. Wang, Compressed video action recognition with dual-stream and dual-modal transformer, IEEE Trans. Circuits Syst. Video Technol. (2023).

[55] W.-J. Yu, Z.-D. Chen, X. Luo, W. Liu, X.-S. Xu, DELTA: A deep dual-stream network for multi-label image classification, Pattern Recognit. 91 (2019) 322–331.

[56] T. Lu, Y. Fang, W. Fu, K. Ding, X. Kang, Dual-stream class-adaptive network for semi-supervised hyperspectral image classification, IEEE Trans. Geosci. Remote Sens. (2024).

[57] Z. Liu, J. Cheng, L. Liu, Z. Ren, Q. Zhang, C. Song, Dual-stream cross-modality fusion transformer for RGB-D action recognition, Knowl.-Based Syst. 255 (2022) 109741.

[58] W. Hu, W. Chen, W. Yuan, Y. Wang, S. Cai, Y. Zhang, Dual-stream pre-training transformer to enhance multimodal learning for social media prediction, in: Proceedings of the 32nd ACM International Conference on Multimedia, 2024, pp. 11450–11456.

[59] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang, Informer: Beyond efficient transformer for long sequence time-series forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 35, 2021, pp. 11106–11115.

[60] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, J. Choo, Reversible instance normalization for accurate time-series forecasting against distribution shift, in: International Conference on Learning Representations, 2021.

[61] Y. Wang, H. Wu, J. Dong, G. Qin, H. Zhang, Y. Liu, Y. Qiu, J. Wang, M. Long, TimeXer: Empowering transformers for time series forecasting with exogenous variables, in: The Thirty-Eighth Annual Conference on Neural Information Processing Systems, 2024.

[62] V. Ekambaram, A. Jati, P. Dayama, S. Mukherjee, N. Nguyen, W.M. Gifford, C. Reddy, J. Kalagnanam, Tiny time mixers (ttms): Fast pre-trained models for enhanced zero/few-shot forecasting of multivariate time series, Adv. Neural Inf. Process. Syst. 37 (2024) 74147–74181.